



University Maintenance System

Full-Stack Web Application

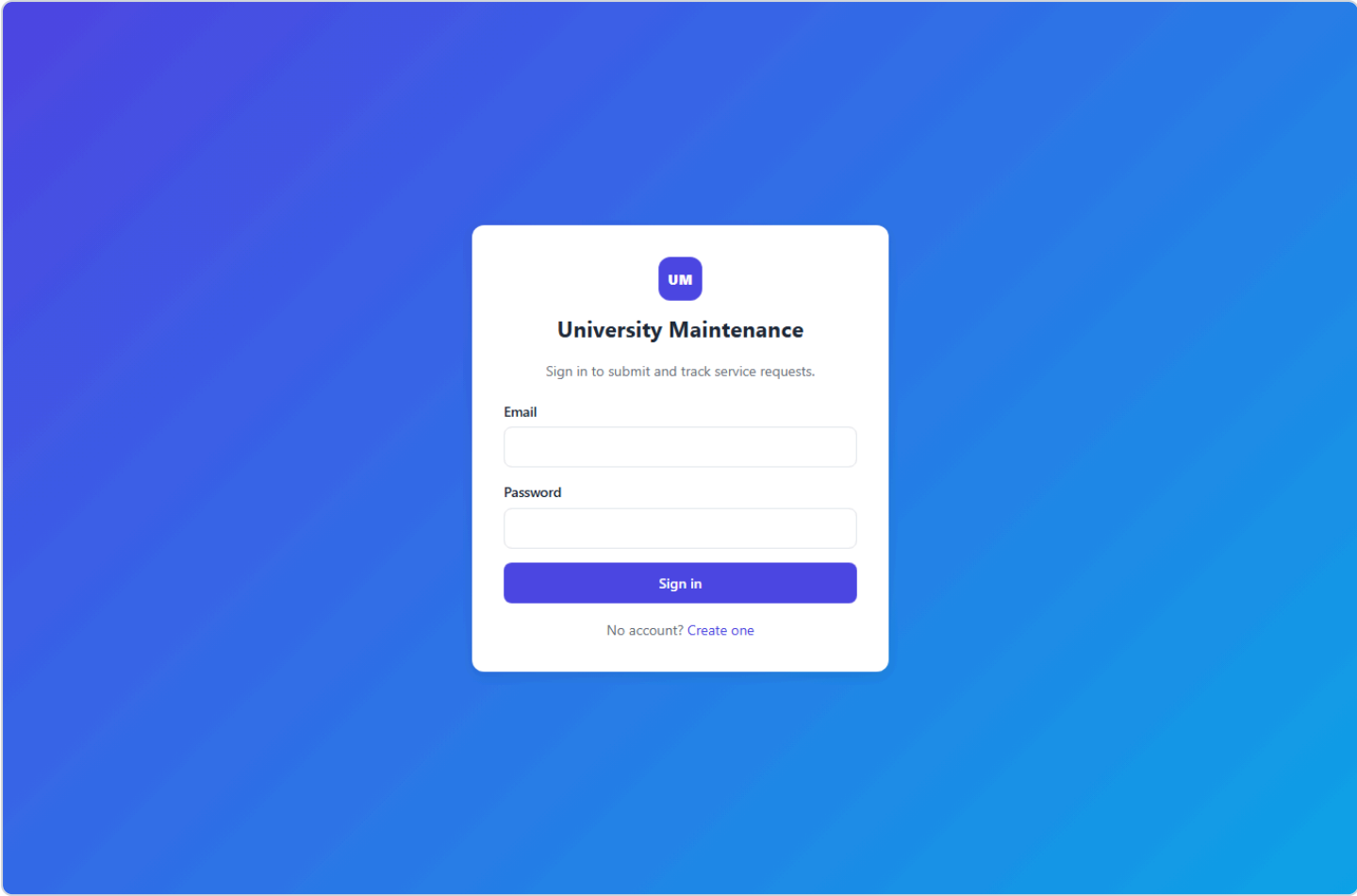
React + TypeScript · .NET 10 Web API · PostgreSQL (EF Core)

Live app: <https://university-maintenance-system-z45k.vercel.app>

API / Swagger: <https://university-maintenance-api.onrender.com/swagger>

Repository: <https://github.com/adebisi4145/UniversityMaintenanceSystem>

Part 1: Application Output (Screenshots)



Login page



Create your account

Register as a student or staff member.

First name

Last name

Email

Password

Create account

Already registered? [Sign in](#)

Registration page (with validation)

UM

UniMaintain

Service Desk

Dashboard

My Requests

Submit Request

S

Sam Student

Student

Sign out

Welcome, Sam👋

Account created!

+ New Request

Submit new maintenance requests and track their progress.

Recent requests

No requests yet.

View all →

Student / staff dashboard

UM

UniMaintain

Service Desk

Dashboard

My Requests

Submit Request

S

Sam Student

Student

Sign out

Submit a Service Request

Title

Flickering light in Lab 3

Category

Electricity

▼

Priority

Medium

▼

Location

Science Block, Lab 3

Description

The ceiling light flickers constantly.

Evidence photo (optional)

Choose File

No file chosen

Cancel

Submit Request

Submit service request form (with evidence upload)

University Maintenance System · Page 5 / 74

UMUniMaintainService Desk

Dashboard

My Requests

Submit Request

S Sam StudentStudent

Sign out

← Back

Request submitted!

Flickering light in Lab 3

Submitted

Medium

Electricity · Science Block, Lab 3

The ceiling light flickers constantly.

REQUESTER

Sam Student

ASSIGNED OFFICER

—

CREATED

04/07/2026, 20:55:28

Activity timeline

Submitted

by Sam Student

04/07/2026, 20:55:28

Request submitted.

Request detail with status / audit timeline

UMUniMaintainService Desk

Dashboard

All Requests

ADMINISTRATION

Users

Categories

Reports

SSystem AdministratorAdmin

Sign out

Welcome, System

Overview of maintenance activity across the university.

1Total Requests

4Total Users

0Completed

0In Progress

Requests by status

Submitted		1
Assigned		0
In Progress		0
Completed		0
Rejected		0

Recent requests

Flickering light in Lab 3

Electricity · Science Block, Lab 3

Submitted

[View all →](#)

Administrator dashboard

UMUniMaintainService Desk

Dashboard

All Requests

ADMINISTRATION

Users

Categories

Reports

SSystem AdministratorAdmin

Sign out

All Requests

Search title, description, location...

All statuses

All categories

TITLE	CATEGORY	LOCATION	PRIORITY	STATUS	REQUESTER
Flickering light in Lab 3	Electricity	Science Block, Lab 3	Medium	Submitted	Sam Student

< Prev

Page 1 of 1 · 1 total

Next >

All requests — search, filter & pagination

UMUniMaintainService Desk

Dashboard

All Requests

ADMINISTRATION

Users

Categories

Reports

SSystem AdministratorAdmin

Sign out

Users

+ New User

Search name or email...

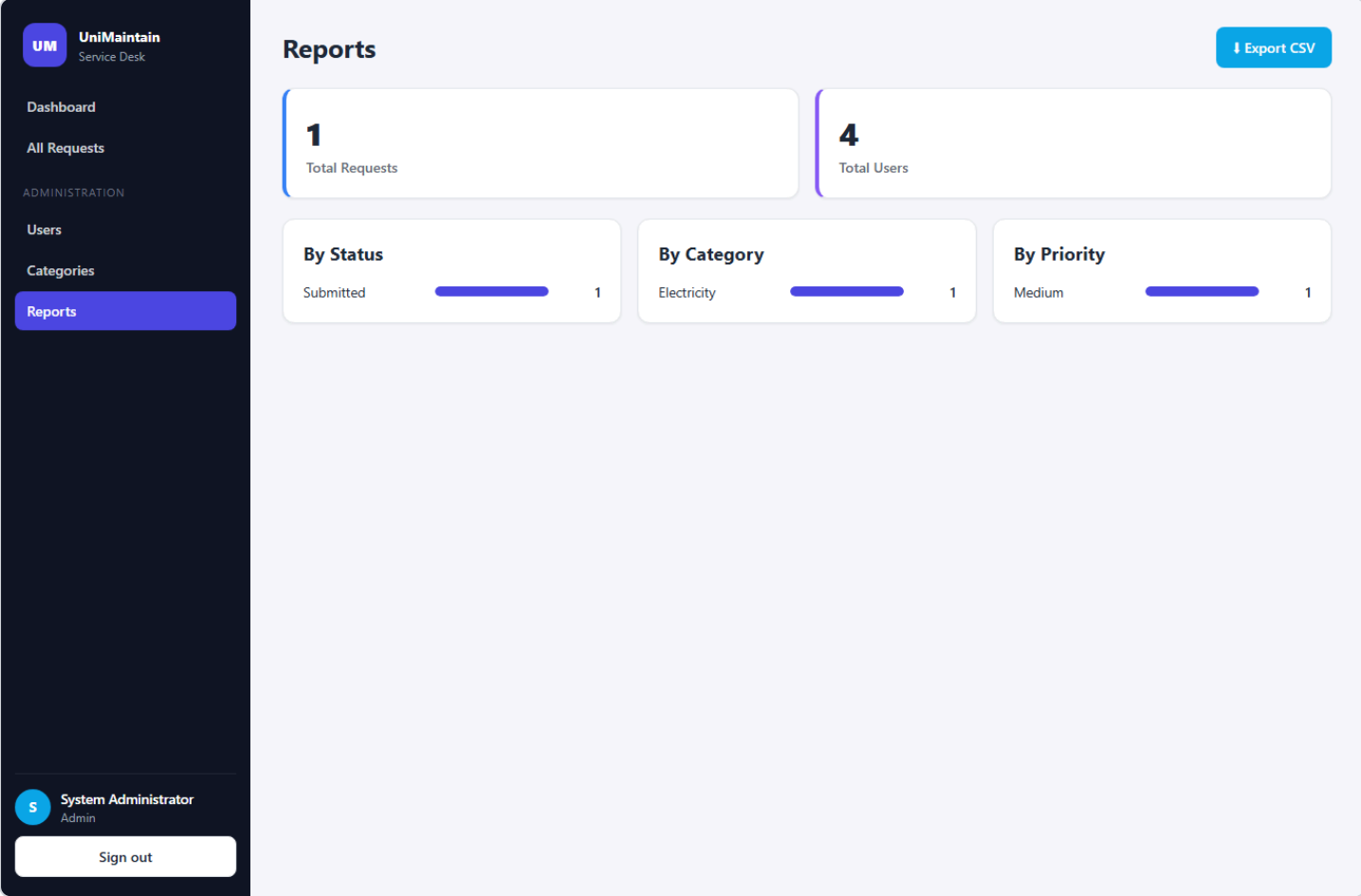
NAME	EMAIL	ROLE	JOINED
Live User	live@student.edu	Student	04/07/2026Delete
Neon Test	neon@student.edu	Student	04/07/2026Delete
Sam Student	sam1783194095458@student.edu	Student	04/07/2026Delete
System Administrator	admin@university.edu	Admin	04/07/2026

< Prev

Page 1 of 1 · 4 total

Next >

User management (admin)



Reports & analytics with CSV export

Part 2: Codebase

Domain layer

UniversityMaintenance.Domain/Common/BaseEntity.cs

C#

```
namespace UniversityMaintenance.Domain.Common;

/// <summary>
/// Base class for all persisted entities. Holds the primary key and audit
/// timestamps so no entity has to repeat them. <see cref="UpdatedAt"/> is
/// stamped centrally in the DbContext on every change.
/// </summary>
public abstract class BaseEntity
{
    public Guid Id { get; set; } = Guid.NewGuid();

    public DateTime CreatedAt { get; set; }

    public DateTime? UpdatedAt { get; set; }
}
```

UniversityMaintenance.Domain/Entities/Assignment.cs

C#

```
using UniversityMaintenance.Domain.Common;

namespace UniversityMaintenance.Domain.Entities;

/// <summary>Links a service request to the maintenance officer responsible for it.</summary>
public class Assignment : BaseEntity
{
    public Guid ServiceRequestId { get; set; }
    public ServiceRequest ServiceRequest { get; set; } = null!;

    /// <summary>The maintenance officer the request is assigned to.</summary>
    public Guid OfficerId { get; set; }
    public User Officer { get; set; } = null!;

    /// <summary>The admin who made the assignment.</summary>
    public Guid AssignedByAdminId { get; set; }

    public DateTime AssignedAt { get; set; }

    public string? Notes { get; set; }
}
```

UniversityMaintenance.Domain/Entities/RequestCategory.cs

C#

```
using UniversityMaintenance.Domain.Common;

namespace UniversityMaintenance.Domain.Entities;

/// <summary>A type of fault, e.g. Electricity, Plumbing, Furniture, Internet, Equipment, Hostel.</summary>
public class RequestCategory : BaseEntity
{
    public string Name { get; set; } = string.Empty;

    public string? Description { get; set; }

    public ICollection<ServiceRequest> ServiceRequests { get; set; } = new List<ServiceRequest>();
}
```

UniversityMaintenance.Domain/Entities/Role.cs

C#

```
using UniversityMaintenance.Domain.Common;

namespace UniversityMaintenance.Domain.Entities;

/// <summary>A security role: Student/Staff, Officer, or Admin. One role has many users.</summary>
public class Role : BaseEntity
{
    public string Name { get; set; } = string.Empty;

    public ICollection<User> Users { get; set; } = new List<User>();
}

/// <summary>Canonical role names used across the app for RBAC.</summary>
public static class RoleNames
{
    public const string Student = "Student";
    public const string Officer = "Officer";
}
```

```

    public const string Admin = "Admin";
}

```

UniversityMaintenance.Domain/Entities/ServiceRequest.cs

C#

```

using UniversityMaintenance.Domain.Common;
using UniversityMaintenance.Domain.Enums;

namespace UniversityMaintenance.Domain.Entities;

/// <summary>A maintenance complaint or service request raised by a student/staff member.</summary>
public class ServiceRequest : BaseEntity
{
    public string Title { get; set; } = string.Empty;

    public string Description { get; set; } = string.Empty;

    public string Location { get; set; } = string.Empty;

    public Priority Priority { get; set; } = Priority.Medium;

    public RequestStatus Status { get; set; } = RequestStatus.Submitted;

    /// <summary>Relative path to the uploaded evidence image, if any.</summary>
    public string? EvidenceImagePath { get; set; }

    public Guid CategoryId { get; set; }
    public RequestCategory Category { get; set; } = null!;

    /// <summary>The user who submitted the request.</summary>
    public Guid RequesterId { get; set; }
    public User Requester { get; set; } = null!;

    /// <summary>Assignment history for this request (typically one active).</summary>
    public ICollection<Assignment> Assignments { get; set; } = new List<Assignment>();

    /// <summary>Audit trail of every status change on this request.</summary>
    public ICollection<StatusUpdate> StatusUpdates { get; set; } = new List<StatusUpdate>();
}

```

UniversityMaintenance.Domain/Entities/StatusUpdate.cs

C#

```

using UniversityMaintenance.Domain.Common;
using UniversityMaintenance.Domain.Enums;

namespace UniversityMaintenance.Domain.Entities;

/// <summary>
/// An entry in a request's activity/audit trail: records who changed the status,
/// from what to what, and any comment.
/// </summary>
public class StatusUpdate : BaseEntity
{
    public Guid ServiceRequestId { get; set; }
    public ServiceRequest ServiceRequest { get; set; } = null!;

    public RequestStatus OldStatus { get; set; }

    public RequestStatus NewStatus { get; set; }

    /// <summary>The user (officer/admin) who made the change.</summary>
    public Guid ChangedById { get; set; }
    public User ChangedBy { get; set; } = null!;

    public string? Comment { get; set; }
}

```

UniversityMaintenance.Domain/Entities/User.cs

C#

```

using UniversityMaintenance.Domain.Common;

namespace UniversityMaintenance.Domain.Entities;

/// <summary>An account that can log in. Each user has exactly one role.</summary>
public class User : BaseEntity
{
    public string FirstName { get; set; } = string.Empty;

    public string LastName { get; set; } = string.Empty;

    /// <summary>Convenience display name; not stored (computed from first + last).</summary>
    public string FullName => $"{FirstName} {LastName}".Trim();

    public string Email { get; set; } = string.Empty;

    public string PasswordHash { get; set; } = string.Empty;

    public Guid RoleId { get; set; }
    public Role Role { get; set; } = null!;
}

```

```

    /// <summary>Requests this user has submitted.</summary>
    public ICollection<ServiceRequest> ServiceRequests { get; set; } = new List<ServiceRequest>();

    /// <summary>Assignments where this user is the maintenance officer.</summary>
    public ICollection<Assignment> Assignments { get; set; } = new List<Assignment>();
}

```

UniversityMaintenance.Domain/Enums/Priority.cs

C#

```

namespace UniversityMaintenance.Domain.Enums;

/// <summary>Urgency of a service request.</summary>
public enum Priority
{
    Low = 0,
    Medium = 1,
    High = 2
}

```

UniversityMaintenance.Domain/Enums/RequestStatus.cs

C#

```

namespace UniversityMaintenance.Domain.Enums;

/// <summary>Lifecycle states a service request moves through.</summary>
public enum RequestStatus
{
    Submitted = 0,
    Assigned = 1,
    InProgress = 2,
    Completed = 3,
    Rejected = 4
}

```

UniversityMaintenance.Domain/UniversityMaintenance.Domain.csproj

XML

```

<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <TargetFramework>net10.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
  </PropertyGroup>

</Project>

```

Application layer

UniversityMaintenance.Application/Common/Exceptions/AppExceptions.cs

C#

```

namespace UniversityMaintenance.Application.Common.Exceptions;

/// <summary>Base class for expected, mappable application errors.</summary>
public abstract class AppException(string message) : Exception(message);

/// <summary>Requested resource does not exist. Maps to HTTP 404.</summary>
public sealed class NotFoundException(string message) : AppException(message);

/// <summary>Caller is authenticated but not allowed to perform the action. Maps to HTTP 403.</summary>
public sealed class ForbiddenException(string message) : AppException(message);

/// <summary>Request conflicts with current state (e.g. duplicate email). Maps to HTTP 409.</summary>
public sealed class ConflictException(string message) : AppException(message);

/// <summary>Bad input that is not covered by model validation. Maps to HTTP 400.</summary>
public sealed class BadRequestException(string message) : AppException(message);

```

UniversityMaintenance.Application/Common/Interfaces/IApplicationDbContext.cs

C#

```

using Microsoft.EntityFrameworkCore;
using UniversityMaintenance.Domain.Entities;

namespace UniversityMaintenance.Application.Common.Interfaces;

/// <summary>
/// Abstraction over the EF Core context so Application services depend on the
/// persistence shape, not on the Infrastructure implementation.
/// </summary>
public interface IApplicationDbContext
{
    DbSet<User> Users { get; }
    DbSet<Role> Roles { get; }
    DbSet<RequestCategory> Categories { get; }
}

```

```

    DbSet<ServiceRequest> ServiceRequests { get; }
    DbSet<Assignment> Assignments { get; }
    DbSet<StatusUpdate> StatusUpdates { get; }

    Task<int> SaveChangesAsync(CancellationToken cancellationToken = default);
}

```

UniversityMaintenance.Application/Common/Interfaces/ICurrentUser.cs

C#

```

namespace UniversityMaintenance.Application.Common.Interfaces;

/// <summary>Exposes the authenticated user's identity, read from the HTTP context.</summary>
public interface ICurrentUser
{
    Guid? UserId { get; }

    string? Email { get; }

    string? Role { get; }

    bool IsInRole(string role);
}

```

UniversityMaintenance.Application/Common/Interfaces/IFileStorageService.cs

C#

```

namespace UniversityMaintenance.Application.Common.Interfaces;

/// <summary>Persists uploaded evidence images and returns a relative path to serve them from.</summary>
public interface IFileStorageService
{
    /// <summary>Validates and saves the file, returning a relative web path (e.g. /uploads/xxx.jpg).</summary>
    Task<string> SaveAsync(Stream content, string fileName, string contentType, long length,
        CancellationToken cancellationToken = default);
}

```

UniversityMaintenance.Application/Common/Interfaces/IJwtTokenService.cs

C#

```

using UniversityMaintenance.Domain.Entities;

namespace UniversityMaintenance.Application.Common.Interfaces;

/// <summary>Issues signed JWT bearer tokens carrying the user's id, email, and role.</summary>
public interface IJwtTokenService
{
    (string Token, DateTime ExpiresAt) CreateToken(User user);
}

```

UniversityMaintenance.Application/Common/Interfaces/IPasswordHasher.cs

C#

```

namespace UniversityMaintenance.Application.Common.Interfaces;

/// <summary>Hashes and verifies passwords. Implemented with ASP.NET Core's PasswordHasher.</summary>
public interface IPasswordHasher
{
    string Hash(string password);

    bool Verify(string hash, string password);
}

```

UniversityMaintenance.Application/Common/Mappings/MappingExtensions.cs

C#

```

using UniversityMaintenance.Application.DTOS;
using UniversityMaintenance.Domain.Entities;

namespace UniversityMaintenance.Application.Common.Mappings;

/// <summary>Manual entity → DTO projections (kept dependency-free, no AutoMapper).</summary>
public static class MappingExtensions
{
    public static UserDto ToDto(this User user) =>
        new(user.Id, user.FirstName, user.LastName, user.FullName, user.Email,
            user.Role?.Name ?? string.Empty, user.CreatedAt);

    public static CategoryDto ToDto(this RequestCategory c) =>
        new(c.Id, c.Name, c.Description);

    public static ServiceRequestDto ToDto(this ServiceRequest r)
    {
        // The most recent assignment identifies the currently responsible officer.
        var current = r.Assignments
            .OrderByDescending(a => a.AssignedAt)
            .FirstOrDefault();

        return new ServiceRequestDto(
            r.Id,

```

```

        r.Title,
        r.Description,
        r.Location,
        r.Priority,
        r.Status,
        r.EvidenceImagePath,
        r.CategoryId,
        r.Category?.Name ?? string.Empty,
        r.RequesterId,
        r.Requester?.FullName ?? string.Empty,
        current?.OfficerId,
        current?.Officer?.FullName,
        r.CreatedAt,
        r.UpdatedAt);
    }

    public static StatusUpdateDto ToDto(this StatusUpdate s) =>
        new(s.Id, s.OldStatus, s.NewStatus, s.ChangedById, s.ChangedBy?.FullName ?? string.Empty,
            s.Comment, s.CreatedAt);

    public static AssignmentDto ToDto(this Assignment a) =>
        new(a.Id, a.ServiceRequestId, a.ServiceRequest?.Title ?? string.Empty, a.OfficerId,
            a.Officer?.FullName ?? string.Empty, a.AssignedByAdminId, a.AssignedAt, a.Notes);
}

```

UniversityMaintenance.Application/Common/Models/PagedResult.cs

C#

```

namespace UniversityMaintenance.Application.Common.Models;

/// <summary>A single page of results plus the paging metadata the client needs.</summary>
public record PagedResult<T>(
    IReadOnlyList<T> Items,
    int Page,
    int PageSize,
    int TotalCount)
{
    public int TotalPages => PageSize <= 0 ? 0 : (int)Math.Ceiling(TotalCount / (double)PageSize);
    public bool HasPrevious => Page > 1;
    public bool HasNext => Page < TotalPages;
}

```

UniversityMaintenance.Application/DTOs/AssignmentDtos.cs

C#

```

namespace UniversityMaintenance.Application.DTOS;

/// <summary>Admin assigns a request to a maintenance officer.</summary>
public class CreateAssignmentDto
{
    public Guid ServiceRequestId { get; set; }
    public Guid OfficerId { get; set; }
    public string? Notes { get; set; }
}

public record AssignmentDto(
    Guid Id,
    Guid ServiceRequestId,
    string ServiceRequestTitle,
    Guid OfficerId,
    string OfficerName,
    Guid AssignedByAdminId,
    DateTime AssignedAt,
    string? Notes);

```

UniversityMaintenance.Application/DTOs/AuthDtos.cs

C#

```

namespace UniversityMaintenance.Application.DTOS;

/// <summary>Public self-registration. New accounts always get the Student role.</summary>
public class RegisterDto
{
    public string FirstName { get; set; } = string.Empty;
    public string LastName { get; set; } = string.Empty;
    public string Email { get; set; } = string.Empty;
    public string Password { get; set; } = string.Empty;
}

public class LoginDto
{
    public string Email { get; set; } = string.Empty;
    public string Password { get; set; } = string.Empty;
}

/// <summary>Returned on successful login/registration.</summary>
public record AuthResponseDto(string Token, DateTime ExpiresAt, UserDto User);

```

UniversityMaintenance.Application/DTOs/CategoryDtos.cs

C#

```
namespace UniversityMaintenance.Application.DTos;

public record CategoryDto(Guid Id, string Name, string? Description);

public class CreateCategoryDto
{
    public string Name { get; set; } = string.Empty;
    public string? Description { get; set; }
}
```

UniversityMaintenance.Application/DTOs/ReportDtos.cs

C#

```
namespace UniversityMaintenance.Application.DTos;

/// <summary>Aggregate counts for the admin dashboard and reports.</summary>
public record ReportSummaryDto(
    int TotalRequests,
    int TotalUsers,
    IReadOnlyDictionary<string, int> RequestsByStatus,
    IReadOnlyDictionary<string, int> RequestsByCategory,
    IReadOnlyDictionary<string, int> RequestsByPriority);
```

UniversityMaintenance.Application/DTOs/ServiceRequestDtos.cs

C#

```
using UniversityMaintenance.Domain.Enums;

namespace UniversityMaintenance.Application.DTos;

/// <summary>Fields for creating a request. The evidence image is uploaded separately as a file.</summary>
public class CreateServiceRequestDto
{
    public string Title { get; set; } = string.Empty;
    public string Description { get; set; } = string.Empty;
    public string Location { get; set; } = string.Empty;
    public Priority Priority { get; set; } = Priority.Medium;
    public Guid CategoryId { get; set; }
}

/// <summary>Officer/Admin moves a request to a new status, optionally with a comment.</summary>
public class UpdateStatusDto
{
    public RequestStatus Status { get; set; }
    public string? Comment { get; set; }
}

/// <summary>Query string parameters for searching, filtering and paging the request list.</summary>
public class ServiceRequestQuery
{
    public string? Search { get; set; }
    public RequestStatus? Status { get; set; }
    public Guid? CategoryId { get; set; }
    public int Page { get; set; } = 1;
    public int PageSize { get; set; } = 10;
}

public record ServiceRequestDto(
    Guid Id,
    string Title,
    string Description,
    string Location,
    Priority Priority,
    RequestStatus Status,
    string? EvidenceImagePath,
    Guid CategoryId,
    string CategoryName,
    Guid RequesterId,
    string RequesterName,
    Guid? AssignedOfficerId,
    string? AssignedOfficerName,
    DateTime CreatedAt,
    DateTime? UpdatedAt);

/// <summary>One entry in a request's status history / audit trail.</summary>
public record StatusUpdateDto(
    Guid Id,
    RequestStatus OldStatus,
    RequestStatus NewStatus,
    Guid ChangedById,
    string ChangedByName,
    string? Comment,
    DateTime CreatedAt);
```

UniversityMaintenance.Application/DTOs/UserDtos.cs

C#

```
namespace UniversityMaintenance.Application.DTos;
```



```

public record UserDto(
    Guid Id, string FirstName, string LastName, string FullName, string Email, string Role, DateTime CreatedAt);

/// <summary>Admin-only creation of Officer/Admin accounts.</summary>
public class CreateUserDto
{
    public string FirstName { get; set; } = string.Empty;
    public string LastName { get; set; } = string.Empty;
    public string Email { get; set; } = string.Empty;
    public string Password { get; set; } = string.Empty;
    public string Role { get; set; } = string.Empty;
}

```

UniversityMaintenance.Application/DependencyInjection.cs

C#

```

using FluentValidation;
using Microsoft.Extensions.DependencyInjection;
using UniversityMaintenance.Application.Services;
using UniversityMaintenance.Application.Services.Interfaces;

namespace UniversityMaintenance.Application;

/// <summary>Registers Application-layer services and validators into the DI container.</summary>
public static class DependencyInjection
{
    public static IServiceCollection AddApplication(this IServiceCollection services)
    {
        services.AddScoped<IAuthService, AuthService>();
        services.AddScoped<IServiceRequestService, ServiceRequestService>();
        services.AddScoped<IAssignmentService, AssignmentService>();
        services.AddScoped<ICategoryService, CategoryService>();
        services.AddScoped<IUserService, UserService>();
        services.AddScoped<IReportService, ReportService>();

        services.AddValidatorsFromAssembly(typeof(DependencyInjection).Assembly);

        return services;
    }
}

```

UniversityMaintenance.Application/Services/AssignmentService.cs

C#

```

using Microsoft.EntityFrameworkCore;
using UniversityMaintenance.Application.Common.Exceptions;
using UniversityMaintenance.Application.Common.Interfaces;
using UniversityMaintenance.Application.Common.Mappings;
using UniversityMaintenance.Application.DTOs;
using UniversityMaintenance.Application.Services.Interfaces;
using UniversityMaintenance.Domain.Entities;
using UniversityMaintenance.Domain.Enums;

namespace UniversityMaintenance.Application.Services;

public class AssignmentService(IApplicationDbContext db, ICurrentUser currentUser)
    : IAssignmentService
{
    public async Task<AssignmentDto> AssignAsync(CreateAssignmentDto dto, CancellationToken ct = default)
    {
        var adminId = currentUser.UserId
            ?? throw new ForbiddenException("You must be signed in.");

        var request = await db.ServiceRequests
            .FirstOrDefaultAsync(r => r.Id == dto.ServiceRequestId, ct)
            ?? throw new NotFoundException("Service request not found.");

        var officer = await db.Users
            .Include(u => u.Role)
            .FirstOrDefaultAsync(u => u.Id == dto.OfficerId, ct)
            ?? throw new NotFoundException("Officer not found.");

        if (officer.Role.Name != RoleNames.Officer)
            throw new BadRequestException("The selected user is not a maintenance officer.");

        var assignment = new Assignment
        {
            ServiceRequestId = request.Id,
            OfficerId = officer.Id,
            Officer = officer,
            ServiceRequest = request,
            AssignedByAdminId = adminId,
            AssignedAt = DateTime.UtcNow,
            Notes = dto.Notes
        };
        db.Assignments.Add(assignment);

        // Move the request forward and record it in the audit trail.
        if (request.Status is RequestStatus.Submitted or RequestStatus.Rejected)
        {
            var old = request.Status;
            request.Status = RequestStatus.Assigned;
            db.StatusUpdates.Add(new StatusUpdate

```

```

        {
            ServiceRequestId = request.Id,
            OldStatus = old,
            NewStatus = RequestStatus.Assigned,
            ChangedById = adminId,
            Comment = $"Assigned to {officer.FullName}."
        });
    }

    await db.SaveChangesAsync(ct);
    return assignment.ToDto();
}
}

```

UniversityMaintenance.Application/Services/AuthService.cs

C#

```

using Microsoft.EntityFrameworkCore;
using UniversityMaintenance.Application.Common.Exceptions;
using UniversityMaintenance.Application.Common.Interfaces;
using UniversityMaintenance.Application.Common.Mappings;
using UniversityMaintenance.Application.DTOS;
using UniversityMaintenance.Application.Services.Interfaces;
using UniversityMaintenance.Domain.Entities;

namespace UniversityMaintenance.Application.Services;

public class AuthService(
    IApplicationDbContext db,
    IPasswordHasher passwordHasher,
    IJwtTokenService jwtTokenService) : IAuthService
{
    public async Task<AuthResponseDto> RegisterAsync(RegisterDto dto, CancellationToken ct = default)
    {
        var email = dto.Email.Trim().ToLowerInvariant();

        if (await db.Users.AnyAsync(u => u.Email == email, ct))
            throw new ConflictException("An account with this email already exists.");

        var studentRole = await db.Roles.FirstOrDefaultAsync(r => r.Name == RoleNames.Student, ct)
            ?? throw new NotFoundException("Default Student role is not configured.");

        var user = new User
        {
            FirstName = dto.FirstName.Trim(),
            LastName = dto.LastName.Trim(),
            Email = email,
            PasswordHash = passwordHasher.Hash(dto.Password),
            RoleId = studentRole.Id,
            Role = studentRole
        };

        db.Users.Add(user);
        await db.SaveChangesAsync(ct);

        return BuildResponse(user);
    }

    public async Task<AuthResponseDto> LoginAsync(LoginDto dto, CancellationToken ct = default)
    {
        var email = dto.Email.Trim().ToLowerInvariant();

        var user = await db.Users
            .Include(u => u.Role)
            .FirstOrDefaultAsync(u => u.Email == email, ct);

        if (user is null || !passwordHasher.Verify(user.PasswordHash, dto.Password))
            throw new BadRequestException("Invalid email or password.");

        return BuildResponse(user);
    }

    private AuthResponseDto BuildResponse(User user)
    {
        var (token, expiresAt) = jwtTokenService.CreateToken(user);
        return new AuthResponseDto(token, expiresAt, user.ToDto());
    }
}

```

UniversityMaintenance.Application/Services/CategoryService.cs

C#

```

using Microsoft.EntityFrameworkCore;
using UniversityMaintenance.Application.Common.Exceptions;
using UniversityMaintenance.Application.Common.Interfaces;
using UniversityMaintenance.Application.Common.Mappings;
using UniversityMaintenance.Application.DTOS;
using UniversityMaintenance.Application.Services.Interfaces;
using UniversityMaintenance.Domain.Entities;

namespace UniversityMaintenance.Application.Services;

public class CategoryService(IApplicationDbContext db) : ICategoryService

```

```

{
    public async Task<IReadOnlyList<CategoryDto>> GetAllAsync(CancellationTok...
    {
        var categories = await db.Categories.OrderBy(c => c.Name).ToListAsync(ct);
        return categories.Select(c => c.ToDto()).ToList();
    }

    public async Task<CategoryDto> CreateAsync(CreateCategoryDto dto, CancellationTok...
    {
        var name = dto.Name.Trim();
        if (await db.Categories.AnyAsync(c => c.Name.ToLower() == name.ToLower(), ct))
            throw new ConflictException("A category with this name already exists.");

        var category = new RequestCategory { Name = name, Description = dto.Description?.Trim() };
        db.Categories.Add(category);
        await db.SaveChangesAsync(ct);
        return category.ToDto();
    }

    public async Task<CategoryDto> UpdateAsync(Guid id, CreateCategoryDto dto, CancellationTok...
    {
        var category = await db.Categories.FirstOrDefaultAsync(c => c.Id == id, ct)
            ?? throw new NotFoundException("Category not found.");

        category.Name = dto.Name.Trim();
        category.Description = dto.Description?.Trim();
        await db.SaveChangesAsync(ct);
        return category.ToDto();
    }

    public async Task DeleteAsync(Guid id, CancellationTok...
    {
        var category = await db.Categories.FirstOrDefaultAsync(c => c.Id == id, ct)
            ?? throw new NotFoundException("Category not found.");

        if (await db.ServiceRequests.AnyAsync(r => r.CategoryId == id, ct))
            throw new ConflictException("Cannot delete a category that has service requests.");

        db.Categories.Remove(category);
        await db.SaveChangesAsync(ct);
    }
}

```

UniversityMaintenance.Application/Services/Interfaces/IAssignmentService.cs

C#

```

using UniversityMaintenance.Application.DTOS;

namespace UniversityMaintenance.Application.Services.Interfaces;

public interface IAssignmentService
{
    Task<AssignmentDto> AssignAsync(CreateAssignmentDto dto, CancellationTok...
}

```

UniversityMaintenance.Application/Services/Interfaces/IAuthService.cs

C#

```

using UniversityMaintenance.Application.DTOS;

namespace UniversityMaintenance.Application.Services.Interfaces;

public interface IAuthService
{
    Task<AuthResponseDto> RegisterAsync(RegisterDto dto, CancellationTok...
    Task<AuthResponseDto> LoginAsync(LoginDto dto, CancellationTok...
}

```

UniversityMaintenance.Application/Services/Interfaces/ICategoryService.cs

C#

```

using UniversityMaintenance.Application.DTOS;

namespace UniversityMaintenance.Application.Services.Interfaces;

public interface ICategoryService
{
    Task<IReadOnlyList<CategoryDto>> GetAllAsync(CancellationTok...
    Task<CategoryDto> CreateAsync(CreateCategoryDto dto, CancellationTok...
    Task<CategoryDto> UpdateAsync(Guid id, CreateCategoryDto dto, CancellationTok...
    Task DeleteAsync(Guid id, CancellationTok...
}

```

UniversityMaintenance.Application/Services/Interfaces/IReportService.cs

C#

```

using UniversityMaintenance.Application.DTOS;

namespace UniversityMaintenance.Application.Services.Interfaces;

public interface IReportService

```

```
{
    Task<ReportSummaryDto> GetSummaryAsync(Cancellation token ct = default);
}
```

UniversityMaintenance.Application/Services/Interfaces/IServiceRequestService.cs

C#

```
using UniversityMaintenance.Application.Common.Models;
using UniversityMaintenance.Application.DTOS;

namespace UniversityMaintenance.Application.Services.Interfaces;

public interface IServiceRequestService
{
    Task<ServiceRequestDto> CreateAsync(CreateServiceRequestDto dto, string? evidenceImagePath,
        Cancellation token ct = default);

    /// <summary>Paged list, scoped by the current user's role (own / assigned / all).</summary>
    Task<PagedResult<ServiceRequestDto>> GetPagedAsync(ServiceRequestQuery query, Cancellation token ct = default);

    Task<ServiceRequestDto> GetByIdAsync(Guid id, Cancellation token ct = default);

    Task<ServiceRequestDto> UpdateStatusAsync(Guid id, UpdateStatusDto dto, Cancellation token ct = default);

    Task<IReadOnlyList<StatusUpdateDto>> GetHistoryAsync(Guid id, Cancellation token ct = default);

    Task DeleteAsync(Guid id, Cancellation token ct = default);
}
```

UniversityMaintenance.Application/Services/Interfaces/IUserService.cs

C#

```
using UniversityMaintenance.Application.Common.Models;
using UniversityMaintenance.Application.DTOS;

namespace UniversityMaintenance.Application.Services.Interfaces;

public interface IUserService
{
    Task<PagedResult<UserDto>> GetPagedAsync(string? search, int page, int pageSize, Cancellation token ct = default);
    Task<IReadOnlyList<UserDto>> GetOfficersAsync(Cancellation token ct = default);
    Task<UserDto> CreateAsync(CreateUserDto dto, Cancellation token ct = default);
    Task DeleteAsync(Guid id, Cancellation token ct = default);
}
```

UniversityMaintenance.Application/Services/ReportService.cs

C#

```
using Microsoft.EntityFrameworkCore;
using UniversityMaintenance.Application.Common.Interfaces;
using UniversityMaintenance.Application.DTOS;
using UniversityMaintenance.Application.Services.Interfaces;

namespace UniversityMaintenance.Application.Services;

public class ReportService(IApplicationDbContext db) : IReportService
{
    public async Task<ReportSummaryDto> GetSummaryAsync(Cancellation token ct = default)
    {
        var totalRequests = await db.ServiceRequests.CountAsync(ct);
        var totalUsers = await db.Users.CountAsync(ct);

        var byStatus = await db.ServiceRequests
            .GroupBy(r => r.Status)
            .Select(g => new { g.Key, Count = g.Count() })
            .ToListAsync(ct);

        var byCategory = await db.ServiceRequests
            .Include(r => r.Category)
            .GroupBy(r => r.Category.Name)
            .Select(g => new { g.Key, Count = g.Count() })
            .ToListAsync(ct);

        var byPriority = await db.ServiceRequests
            .GroupBy(r => r.Priority)
            .Select(g => new { g.Key, Count = g.Count() })
            .ToListAsync(ct);

        return new ReportSummaryDto(
            totalRequests,
            totalUsers,
            byStatus.ToDictionary(x => x.Key.ToString(), x => x.Count),
            byCategory.ToDictionary(x => x.Key, x => x.Count),
            byPriority.ToDictionary(x => x.Key.ToString(), x => x.Count));
    }
}
```

```

using Microsoft.EntityFrameworkCore;
using UniversityMaintenance.Application.Common.Exceptions;
using UniversityMaintenance.Application.Common.Interfaces;
using UniversityMaintenance.Application.Common.Mappings;
using UniversityMaintenance.Application.Common.Models;
using UniversityMaintenance.Application.DTOs;
using UniversityMaintenance.Application.Services.Interfaces;
using UniversityMaintenance.Domain.Entities;
using UniversityMaintenance.Domain.Enums;

namespace UniversityMaintenance.Application.Services;

public class ServiceRequestService(IApplicationDbContext db, ICurrentUser currentUser)
    : IServiceRequestService
{
    private Guid UserId => currentUser.UserId
        ?? throw new ForbiddenException("You must be signed in.");

    public async Task<ServiceRequestDto> CreateAsync(CreateServiceRequestDto dto,
        string? evidenceImagePath, CancellationToken ct = default)
    {
        var categoryExists = await db.Categories.AnyAsync(c => c.Id == dto.CategoryId, ct);
        if (!categoryExists)
            throw new BadRequestException("The selected category does not exist.");

        var request = new ServiceRequest
        {
            Title = dto.Title.Trim(),
            Description = dto.Description.Trim(),
            Location = dto.Location.Trim(),
            Priority = dto.Priority,
            Status = RequestStatus.Submitted,
            EvidenceImagePath = evidenceImagePath,
            CategoryId = dto.CategoryId,
            RequesterId = UserId
        };

        db.ServiceRequests.Add(request);

        // Seed the audit trail with the initial submission.
        db.StatusUpdates.Add(new StatusUpdate
        {
            ServiceRequestId = request.Id,
            OldStatus = RequestStatus.Submitted,
            NewStatus = RequestStatus.Submitted,
            ChangedById = UserId,
            Comment = "Request submitted."
        });

        await db.SaveChangesAsync(ct);
        return await GetByIdAsync(request.Id, ct);
    }

    public async Task<PagedResult<ServiceRequestDto>> GetPagedAsync(ServiceRequestQuery query,
        CancellationToken ct = default)
    {
        var q = BaseQuery();

        // Role-based scoping: students see their own, officers see assigned, admins see all.
        if (currentUser.IsInRole(RoleNames.Student))
            q = q.Where(r => r.RequesterId == UserId);
        else if (currentUser.IsInRole(RoleNames.Officer))
            q = q.Where(r => r.Assignments.Any(a => a.OfficerId == UserId));

        if (!string.IsNullOrWhiteSpace(query.Search))
        {
            var term = query.Search.Trim().ToLower();
            q = q.Where(r =>
                r.Title.ToLower().Contains(term) ||
                r.Description.ToLower().Contains(term) ||
                r.Location.ToLower().Contains(term));
        }

        if (query.Status.HasValue)
            q = q.Where(r => r.Status == query.Status.Value);

        if (query.CategoryId.HasValue)
            q = q.Where(r => r.CategoryId == query.CategoryId.Value);

        var page = query.Page < 1 ? 1 : query.Page;
        var pageSize = query.PageSize is < 1 or > 100 ? 10 : query.PageSize;

        var total = await q.CountAsync(ct);

        var items = await q
            .OrderByDescending(r => r.CreatedAt)
            .Skip((page - 1) * pageSize)
            .Take(pageSize)
            .ToListAsync(ct);

        return new PagedResult<ServiceRequestDto>(
            items.Select(r => r.ToDto()).ToList(), page, pageSize, total);
    }
}

```

```

    }

    public async Task<ServiceRequestDto> GetByIdAsync(Guid id, CancellationToken ct = default)
    {
        var request = await BaseQuery().FirstOrDefaultAsync(r => r.Id == id, ct)
            ?? throw new NotFoundException("Service request not found.");

        EnsureCanView(request);
        return request.ToDto();
    }

    public async Task<ServiceRequestDto> UpdateStatusAsync(Guid id, UpdateStatusDto dto,
        CancellationToken ct = default)
    {
        var request = await BaseQuery().FirstOrDefaultAsync(r => r.Id == id, ct)
            ?? throw new NotFoundException("Service request not found.");

        // Only the assigned officer or an admin may change status.
        var isAssignedOfficer = currentUser.IsInRole(RoleNames.Officer)
            && request.Assignments.Any(a => a.OfficerId == UserId);
        if (!currentUser.IsInRole(RoleNames.Admin) && !isAssignedOfficer)
            throw new ForbiddenException("You are not allowed to update this request.");

        if (dto.Status == request.Status)
            throw new BadRequestException("The request is already in that status.");

        var old = request.Status;
        request.Status = dto.Status;

        db.StatusUpdates.Add(new StatusUpdate
        {
            ServiceRequestId = request.Id,
            OldStatus = old,
            NewStatus = dto.Status,
            ChangedById = UserId,
            Comment = dto.Comment
        });

        await db.SaveChangesAsync(ct);
        return await GetByIdAsync(request.Id, ct);
    }

    public async Task<IReadOnlyList<StatusUpdateDto>> GetHistoryAsync(Guid id,
        CancellationToken ct = default)
    {
        var request = await BaseQuery().FirstOrDefaultAsync(r => r.Id == id, ct)
            ?? throw new NotFoundException("Service request not found.");

        EnsureCanView(request);

        var history = await db.StatusUpdates
            .Include(s => s.ChangedBy)
            .Where(s => s.ServiceRequestId == id)
            .OrderBy(s => s.CreatedAt)
            .ToListAsync(ct);

        return history.Select(s => s.ToDto()).ToList();
    }

    public async Task DeleteAsync(Guid id, CancellationToken ct = default)
    {
        var request = await db.ServiceRequests.FirstOrDefaultAsync(r => r.Id == id, ct)
            ?? throw new NotFoundException("Service request not found.");

        // Admins may delete any request; a requester may delete their own while still Submitted.
        var isOwnerAndPending = request.RequesterId == UserId
            && request.Status == RequestStatus.Submitted;
        if (!currentUser.IsInRole(RoleNames.Admin) && !isOwnerAndPending)
            throw new ForbiddenException("You are not allowed to delete this request.");

        db.ServiceRequests.Remove(request);
        await db.SaveChangesAsync(ct);
    }

    private IQueryable<ServiceRequest> BaseQuery() =>
        db.ServiceRequests
            .Include(r => r.Category)
            .Include(r => r.Requester)
            .Include(r => r.Assignments).ThenInclude(a => a.Officer)
            .AsQueryable();

    private void EnsureCanView(ServiceRequest request)
    {
        if (currentUser.IsInRole(RoleNames.Admin))
            return;
        if (currentUser.IsInRole(RoleNames.Officer)
            && request.Assignments.Any(a => a.OfficerId == UserId))
            return;
        if (request.RequesterId == UserId)
            return;

        throw new ForbiddenException("You are not allowed to view this request.");
    }
}

```

```

using Microsoft.EntityFrameworkCore;
using UniversityMaintenance.Application.Common.Exceptions;
using UniversityMaintenance.Application.Common.Interfaces;
using UniversityMaintenance.Application.Common.Mappings;
using UniversityMaintenance.Application.Common.Models;
using UniversityMaintenance.Application.DTOS;
using UniversityMaintenance.Application.Services.Interfaces;
using UniversityMaintenance.Domain.Entities;

namespace UniversityMaintenance.Application.Services;

public class UserService(
    IApplicationDbContext db,
    IPasswordHasher passwordHasher,
    ICurrentUser currentUser) : IUserService
{
    private static readonly string[] AssignableRoles =
        [RoleNames.Student, RoleNames.Officer, RoleNames.Admin];

    public async Task<PagedResult<UserDto>> GetPagedAsync(string? search, int page, int pageSize,
        CancellationToken ct = default)
    {
        page = page < 1 ? 1 : page;
        pageSize = pageSize is < 1 or > 100 ? 10 : pageSize;

        var q = db.Users.Include(u => u.Role).AsQueryable();

        if (!string.IsNullOrEmpty(search))
        {
            var term = search.Trim().ToLower();
            q = q.Where(u => u.FullName.ToLower().Contains(term) || u.Email.ToLower().Contains(term));
        }

        var total = await q.CountAsync(ct);
        var users = await q
            .OrderBy(u => u.FirstName)
            .ThenBy(u => u.LastName)
            .Skip((page - 1) * pageSize)
            .Take(pageSize)
            .ToListAsync(ct);

        return new PagedResult<UserDto>(users.Select(u => u.ToDto()).ToList(), page, pageSize, total);
    }

    public async Task<IReadOnlyList<UserDto>> GetOfficersAsync(CancellationToken ct = default)
    {
        var officers = await db.Users
            .Include(u => u.Role)
            .Where(u => u.Role.Name == RoleNames.Officer)
            .OrderBy(u => u.FirstName)
            .ThenBy(u => u.LastName)
            .ToListAsync(ct);

        return officers.Select(u => u.ToDto()).ToList();
    }

    public async Task<UserDto> CreateAsync(CreateUserDto dto, CancellationToken ct = default)
    {
        if (!AssignableRoles.Contains(dto.Role))
            throw new BadRequestException("Invalid role.");

        var email = dto.Email.Trim().ToLowerInvariant();
        if (await db.Users.AnyAsync(u => u.Email == email, ct))
            throw new ConflictException("An account with this email already exists.");

        var role = await db.Roles.FirstOrDefaultAsync(r => r.Name == dto.Role, ct)
            ?? throw new NotFoundException("Role not found.");

        var user = new User
        {
            FirstName = dto.FirstName.Trim(),
            LastName = dto.LastName.Trim(),
            Email = email,
            PasswordHash = passwordHasher.Hash(dto.Password),
            RoleId = role.Id,
            Role = role
        };
        db.Users.Add(user);
        await db.SaveChangesAsync(ct);
        return user.ToDto();
    }

    public async Task DeleteAsync(Guid id, CancellationToken ct = default)
    {
        if (currentUser.UserId == id)
            throw new BadRequestException("You cannot delete your own account.");

        var user = await db.Users.FirstOrDefaultAsync(u => u.Id == id, ct)
            ?? throw new NotFoundException("User not found.");

        db.Users.Remove(user);
        await db.SaveChangesAsync(ct);
    }
}

```

```
}  
}
```

UniversityMaintenance.Application/UniversityMaintenance.Application.csproj

XML

```
<Project Sdk="Microsoft.NET.Sdk">  
  
  <ItemGroup>  
    <ProjectReference Include="..\UniversityMaintenance.Domain\UniversityMaintenance.Domain.csproj" />  
  </ItemGroup>  
  
  <ItemGroup>  
    <PackageReference Include="FluentValidation" Version="12.1.1" />  
    <PackageReference Include="FluentValidation.DependencyInjectionExtensions" Version="12.1.1" />  
    <PackageReference Include="Microsoft.EntityFrameworkCore" Version="10.0.9" />  
    <PackageReference Include="Microsoft.Extensions.DependencyInjection.Abstractions" Version="10.0.9" />  
  </ItemGroup>  
  
  <PropertyGroup>  
    <TargetFramework>net10.0</TargetFramework>  
    <ImplicitUsings>enable</ImplicitUsings>  
    <Nullable>enable</Nullable>  
  </PropertyGroup>  
  
</Project>
```

UniversityMaintenance.Application/Validators/AuthValidators.cs

C#

```
using FluentValidation;  
using UniversityMaintenance.Application.DTOS;  
  
namespace UniversityMaintenance.Application.Validators;  
  
public class RegisterDtoValidator : AbstractValidator<RegisterDto>  
{  
    public RegisterDtoValidator()  
    {  
        RuleFor(x => x.FirstName).NotEmpty().MaximumLength(75);  
        RuleFor(x => x.LastName).NotEmpty().MaximumLength(75);  
        RuleFor(x => x.Email).NotEmpty().EmailAddress().MaximumLength(256);  
        RuleFor(x => x.Password).NotEmpty().MinimumLength(6).MaximumLength(100);  
    }  
}  
  
public class LoginDtoValidator : AbstractValidator<LoginDto>  
{  
    public LoginDtoValidator()  
    {  
        RuleFor(x => x.Email).NotEmpty().EmailAddress();  
        RuleFor(x => x.Password).NotEmpty();  
    }  
}  
  
public class CreateUserDtoValidator : AbstractValidator<CreateUserDto>  
{  
    public CreateUserDtoValidator()  
    {  
        RuleFor(x => x.FirstName).NotEmpty().MaximumLength(75);  
        RuleFor(x => x.LastName).NotEmpty().MaximumLength(75);  
        RuleFor(x => x.Email).NotEmpty().EmailAddress().MaximumLength(256);  
        RuleFor(x => x.Password).NotEmpty().MinimumLength(6).MaximumLength(100);  
        RuleFor(x => x.Role).NotEmpty();  
    }  
}
```

UniversityMaintenance.Application/Validators/ServiceRequestValidators.cs

C#

```
using FluentValidation;  
using UniversityMaintenance.Application.DTOS;  
  
namespace UniversityMaintenance.Application.Validators;  
  
public class CreateServiceRequestDtoValidator : AbstractValidator<CreateServiceRequestDto>  
{  
    public CreateServiceRequestDtoValidator()  
    {  
        RuleFor(x => x.Title).NotEmpty().MaximumLength(200);  
        RuleFor(x => x.Description).NotEmpty().MaximumLength(2000);  
        RuleFor(x => x.Location).NotEmpty().MaximumLength(200);  
        RuleFor(x => x.Priority).IsInEnum();  
        RuleFor(x => x.CategoryId).NotEmpty();  
    }  
}  
  
public class UpdateStatusDtoValidator : AbstractValidator<UpdateStatusDto>  
{  
    public UpdateStatusDtoValidator()  
    {  
        RuleFor(x => x.Status).IsInEnum();  
    }  
}
```



```

        RuleFor(x => x.Comment).HasMaxLength(1000);
    }
}

public class CreateAssignmentDtoValidator : AbstractValidator<CreateAssignmentDto>
{
    public CreateAssignmentDtoValidator()
    {
        RuleFor(x => x.ServiceRequestId).NotEmpty();
        RuleFor(x => x.OfficerId).NotEmpty();
        RuleFor(x => x.Notes).HasMaxLength(1000);
    }
}

public class CreateCategoryDtoValidator : AbstractValidator<CreateCategoryDto>
{
    public CreateCategoryDtoValidator()
    {
        RuleFor(x => x.Name).NotEmpty().HasMaxLength(100);
        RuleFor(x => x.Description).HasMaxLength(500);
    }
}

```

Infrastructure layer

UniversityMaintenance.Infrastructure/DependencyInjection.cs

C#

```

using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using UniversityMaintenance.Application.Common.Interfaces;
using UniversityMaintenance.Infrastructure.Persistence;
using UniversityMaintenance.Infrastructure.Security;
using UniversityMaintenance.Infrastructure.Storage;

namespace UniversityMaintenance.Infrastructure;

/// <summary>Registers EF Core, security, and storage services into the DI container.</summary>
public static class DependencyInjection
{
    public static IServiceCollection AddInfrastructure(
        this IServiceCollection services, IConfiguration configuration)
    {
        var connectionString = configuration.GetConnectionString("DefaultConnection")
            ?? throw new InvalidOperationException("Connection string 'DefaultConnection' is not configured.");

        services.AddDbContext<AppDbContext>(options =>
            options.UseNpgsql(connectionString));

        services.AddScoped<IApplicationDbContext>(sp => sp.GetRequiredService<AppDbContext>());

        services.Configure<JwtSettings>(configuration.GetSection(JwtSettings.SectionName));
        services.Configure<FileStorageOptions>(configuration.GetSection(FileStorageOptions.SectionName));

        services.AddSingleton<IPasswordHasher, PasswordHasherService>();
        services.AddScoped<IJwtTokenService, JwtTokenService>();
        services.AddScoped<IFileStorageService, FileStorageService>();

        return services;
    }
}

```

UniversityMaintenance.Infrastructure/Persistence/AppDbContext.cs

C#

```

using Microsoft.EntityFrameworkCore;
using UniversityMaintenance.Application.Common.Interfaces;
using UniversityMaintenance.Domain.Common;
using UniversityMaintenance.Domain.Entities;

namespace UniversityMaintenance.Infrastructure.Persistence;

public class AppDbContext(DbContextOptions<AppDbContext> options)
    : DbContext(options), IApplicationDbContext
{
    public DbSet<User> Users => Set<User>();
    public DbSet<Role> Roles => Set<Role>();
    public DbSet<RequestCategory> Categories => Set<RequestCategory>();
    public DbSet<ServiceRequest> ServiceRequests => Set<ServiceRequest>();
    public DbSet<Assignment> Assignments => Set<Assignment>();
    public DbSet<StatusUpdate> StatusUpdates => Set<StatusUpdate>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);
        modelBuilder.ApplyConfigurationsFromAssembly(typeof(AppDbContext).Assembly);
    }

    public override Task<int> SaveChangesAsync(CancellationToken cancellationToken = default)
    {

```

```

        StampAudit();
        return base.SaveChangesAsync(cancellationToken);
    }

    public override int SaveChanges()
    {
        StampAudit();
        return base.SaveChanges();
    }

    /// <summary>Centrally sets CreatedAt on insert and UpdatedAt on modify for every entity.</summary>
    private void StampAudit()
    {
        var now = DateTime.UtcNow;
        foreach (var entry in ChangeTracker.Entries<BaseEntity>())
        {
            switch (entry.State)
            {
                case EntityState.Added:
                    entry.Entity.CreatedAt = now;
                    break;
                case EntityState.Modified:
                    entry.Entity.UpdatedAt = now;
                    break;
            }
        }
    }
}

```

UniversityMaintenance.Infrastructure/Persistence/AppDbContextFactory.cs

C#

```

using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Design;

namespace UniversityMaintenance.Infrastructure.Persistence;

/// <summary>
/// Used only by the EF Core CLI (migrations/scaffolding) so it can build the context
/// without running the API host. The connection string here is never used to talk to a
/// live database – only to select the Npgsql provider.
/// </summary>
public class AppDbContextFactory : IDesignTimeDbContextFactory<AppDbContext>
{
    public AppDbContext CreateDbContext(string[] args)
    {
        var connectionString = Environment.GetEnvironmentVariable("DESIGN_TIME_CONNECTION")
            ?? "Host=localhost;Port=5432;Database=university_maintenance;Username=postgres;Password=postgres";

        var options = new DbContextOptionsBuilder<AppDbContext>()
            .UseNpgsql(connectionString)
            .Options;

        return new AppDbContext(options);
    }
}

```

UniversityMaintenance.Infrastructure/Persistence/Configurations/EntityConfigurations.cs

C#

```

using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Metadata.Builders;
using UniversityMaintenance.Domain.Entities;

namespace UniversityMaintenance.Infrastructure.Persistence.Configurations;

public class RoleConfiguration : IEntityTypeConfiguration<Role>
{
    public void Configure(EntityTypeBuilder<Role> builder)
    {
        builder.Property(r => r.Name).IsRequired().HasMaxLength(50);
        builder.HasIndex(r => r.Name).IsUnique();

        builder.HasMany(r => r.Users)
            .WithOne(u => u.Role)
            .HasForeignKey(u => u.RoleId)
            .OnDelete(DeleteBehavior.Restrict);
    }
}

public class UserConfiguration : IEntityTypeConfiguration<User>
{
    public void Configure(EntityTypeBuilder<User> builder)
    {
        builder.Property(u => u.FirstName).IsRequired().HasMaxLength(75);
        builder.Property(u => u.LastName).IsRequired().HasMaxLength(75);
        builder.Property(u => u.Email).IsRequired().HasMaxLength(256);
        builder.Property(u => u.PasswordHash).IsRequired();
        builder.HasIndex(u => u.Email).IsUnique();

        // FullName is derived from FirstName + LastName; do not persist it.
        builder.Ignore(u => u.FullName);
    }
}

```

```

}

public class RequestCategoryConfiguration : IEntityTypeConfiguration<RequestCategory>
{
    public void Configure(EntityTypeBuilder<RequestCategory> builder)
    {
        builder.Property(c => c.Name).IsRequired().HasMaxLength(100);
        builder.Property(c => c.Description).HasMaxLength(500);
        builder.HasIndex(c => c.Name).IsUnique();
    }
}

public class ServiceRequestConfiguration : IEntityTypeConfiguration<ServiceRequest>
{
    public void Configure(EntityTypeBuilder<ServiceRequest> builder)
    {
        builder.Property(r => r.Title).IsRequired().HasMaxLength(200);
        builder.Property(r => r.Description).IsRequired().HasMaxLength(2000);
        builder.Property(r => r.Location).IsRequired().HasMaxLength(200);
        builder.Property(r => r.EvidenceImagePath).HasMaxLength(400);
        builder.Property(r => r.Status).HasConversion<string>().HasMaxLength(20);
        builder.Property(r => r.Priority).HasConversion<string>().HasMaxLength(20);

        builder.HasOne(r => r.Category)
            .WithMany(c => c.ServiceRequests)
            .HasForeignKey(r => r.CategoryId)
            .OnDelete(DeleteBehavior.Restrict);

        builder.HasOne(r => r.Requester)
            .WithMany(u => u.ServiceRequests)
            .HasForeignKey(r => r.RequesterId)
            .OnDelete(DeleteBehavior.Restrict);
    }
}

public class AssignmentConfiguration : IEntityTypeConfiguration<Assignment>
{
    public void Configure(EntityTypeBuilder<Assignment> builder)
    {
        builder.Property(a => a.Notes).HasMaxLength(1000);

        builder.HasOne(a => a.ServiceRequest)
            .WithMany(r => r.Assignments)
            .HasForeignKey(a => a.ServiceRequestId)
            .OnDelete(DeleteBehavior.Cascade);

        builder.HasOne(a => a.Officer)
            .WithMany(u => u.Assignments)
            .HasForeignKey(a => a.OfficerId)
            .OnDelete(DeleteBehavior.Restrict);
    }
}

public class StatusUpdateConfiguration : IEntityTypeConfiguration<StatusUpdate>
{
    public void Configure(EntityTypeBuilder<StatusUpdate> builder)
    {
        builder.Property(s => s.Comment).HasMaxLength(1000);
        builder.Property(s => s.OldStatus).HasConversion<string>().HasMaxLength(20);
        builder.Property(s => s.NewStatus).HasConversion<string>().HasMaxLength(20);

        builder.HasOne(s => s.ServiceRequest)
            .WithMany(r => r.StatusUpdates)
            .HasForeignKey(s => s.ServiceRequestId)
            .OnDelete(DeleteBehavior.Cascade);

        builder.HasOne(s => s.ChangedBy)
            .WithMany()
            .HasForeignKey(s => s.ChangedById)
            .OnDelete(DeleteBehavior.Restrict);
    }
}

```

UniversityMaintenance.Infrastructure/Persistence/DbSeeder.cs

C#

```

using Microsoft.EntityFrameworkCore;
using UniversityMaintenance.Application.Common.Interfaces;
using UniversityMaintenance.Domain.Entities;

namespace UniversityMaintenance.Infrastructure.Persistence;

/// <summary>Seeds the baseline data every deployment needs: roles, categories, and a default admin.</summary>
public static class DbSeeder
{
    private static readonly (string Name, string Description)[] DefaultCategories =
    [
        ("Electricity", "Faulty electricity, power sockets, lighting"),
        ("Plumbing", "Leaking pipes, taps, drainage, toilets"),
        ("Furniture", "Damaged desks, chairs, beds, cupboards"),
        ("Internet", "Wi-Fi and network connectivity problems"),
        ("Equipment", "Classroom and lab equipment issues"),
        ("Hostel", "General hostel maintenance complaints")
    ];
}

```

```

public static async Task SeedAsync(
    AppDbContext db,
    IPasswordHasher passwordHasher,
    string adminEmail,
    string adminPassword,
    CancellationToken ct = default)
{
    // Apply migrations against a real relational database; fall back to EnsureCreated
    // for the in-memory provider used in tests.
    var isInMemory = db.Database.ProviderName == "Microsoft.EntityFrameworkCore.InMemory";
    if (isInMemory)
        await db.Database.EnsureCreatedAsync(ct);
    else
        await db.Database.MigrateAsync(ct);

    // Roles
    foreach (var name in new[] { RoleNames.Student, RoleNames.Officer, RoleNames.Admin })
    {
        if (!await db.Roles.AnyAsync(r => r.Name == name, ct))
            db.Roles.Add(new Role { Name = name });
    }
    await db.SaveChangesAsync(ct);

    // Categories
    foreach (var (catName, description) in DefaultCategories)
    {
        if (!await db.Categories.AnyAsync(c => c.Name == catName, ct))
            db.Categories.Add(new RequestCategory { Name = catName, Description = description });
    }
    await db.SaveChangesAsync(ct);

    // Default admin account
    var email = adminEmail.Trim().ToLowerInvariant();
    if (!await db.Users.AnyAsync(u => u.Email == email, ct))
    {
        var adminRole = await db.Roles.FirstAsync(r => r.Name == RoleNames.Admin, ct);
        db.Users.Add(new User
        {
            FirstName = "System",
            LastName = "Administrator",
            Email = email,
            PasswordHash = passwordHasher.Hash(adminPassword),
            RoleId = adminRole.Id
        });
        await db.SaveChangesAsync(ct);
    }
}
}

```

UniversityMaintenance.Infrastructure/Persistence/Migrations/20260704121559_InitialCreate.cs

C#

```

using System;
using Microsoft.EntityFrameworkCore.Migrations;

#nullable disable

namespace UniversityMaintenance.Infrastructure.Persistence.Migrations
{
    /// <inheritdoc />
    public partial class InitialCreate : Migration
    {
        /// <inheritdoc />
        protected override void Up(MigrationBuilder migrationBuilder)
        {
            migrationBuilder.CreateTable(
                name: "Categories",
                columns: table => new
                {
                    Id = table.Column<Guid>(type: "uuid", nullable: false),
                    Name = table.Column<string>(type: "character varying(100)", maxLength: 100, nullable: false),
                    Description = table.Column<string>(type: "character varying(500)", maxLength: 500, nullable: true),
                    CreatedAt = table.Column<DateTime>(type: "timestamp with time zone", nullable: false),
                    UpdatedAt = table.Column<DateTime>(type: "timestamp with time zone", nullable: true)
                },
                constraints: table =>
                {
                    table.PrimaryKey("PK_Categories", x => x.Id);
                });

            migrationBuilder.CreateTable(
                name: "Roles",
                columns: table => new
                {
                    Id = table.Column<Guid>(type: "uuid", nullable: false),
                    Name = table.Column<string>(type: "character varying(50)", maxLength: 50, nullable: false),
                    CreatedAt = table.Column<DateTime>(type: "timestamp with time zone", nullable: false),
                    UpdatedAt = table.Column<DateTime>(type: "timestamp with time zone", nullable: true)
                },
                constraints: table =>
                {
                    table.PrimaryKey("PK_Roles", x => x.Id);
                });
        }
    }
}

```

```

migrationBuilder.CreateTable(
    name: "Users",
    columns: table => new
    {
        Id = table.Column<Guid>(type: "uuid", nullable: false),
        FirstName = table.Column<string>(type: "character varying(75)", maxLength: 75, nullable: false),
        LastName = table.Column<string>(type: "character varying(75)", maxLength: 75, nullable: false),
        Email = table.Column<string>(type: "character varying(256)", maxLength: 256, nullable: false),
        PasswordHash = table.Column<string>(type: "text", nullable: false),
        RoleId = table.Column<Guid>(type: "uuid", nullable: false),
        CreatedAt = table.Column<DateTime>(type: "timestamp with time zone", nullable: false),
        UpdatedAt = table.Column<DateTime>(type: "timestamp with time zone", nullable: true)
    },
    constraints: table =>
    {
        table.PrimaryKey("PK_Users", x => x.Id);
        table.ForeignKey(
            name: "FK_Users_Roles_RoleId",
            column: x => x.RoleId,
            principalTable: "Roles",
            principalColumn: "Id",
            onDelete: ReferentialAction.Restrict);
    });

migrationBuilder.CreateTable(
    name: "ServiceRequests",
    columns: table => new
    {
        Id = table.Column<Guid>(type: "uuid", nullable: false),
        Title = table.Column<string>(type: "character varying(200)", maxLength: 200, nullable: false),
        Description = table.Column<string>(type: "character varying(2000)", maxLength: 2000, nullable: false),
        Location = table.Column<string>(type: "character varying(200)", maxLength: 200, nullable: false),
        Priority = table.Column<string>(type: "character varying(20)", maxLength: 20, nullable: false),
        Status = table.Column<string>(type: "character varying(20)", maxLength: 20, nullable: false),
        EvidenceImagePath = table.Column<string>(type: "character varying(400)", maxLength: 400, nullable: true),
        CategoryId = table.Column<Guid>(type: "uuid", nullable: false),
        RequesterId = table.Column<Guid>(type: "uuid", nullable: false),
        CreatedAt = table.Column<DateTime>(type: "timestamp with time zone", nullable: false),
        UpdatedAt = table.Column<DateTime>(type: "timestamp with time zone", nullable: true)
    },
    constraints: table =>
    {
        table.PrimaryKey("PK_ServiceRequests", x => x.Id);
        table.ForeignKey(
            name: "FK_ServiceRequests_Categories_CategoryId",
            column: x => x.CategoryId,
            principalTable: "Categories",
            principalColumn: "Id",
            onDelete: ReferentialAction.Restrict);
        table.ForeignKey(
            name: "FK_ServiceRequests_Users_RequesterId",
            column: x => x.RequesterId,
            principalTable: "Users",
            principalColumn: "Id",
            onDelete: ReferentialAction.Restrict);
    });

migrationBuilder.CreateTable(
    name: "Assignments",
    columns: table => new
    {
        Id = table.Column<Guid>(type: "uuid", nullable: false),
        ServiceRequestId = table.Column<Guid>(type: "uuid", nullable: false),
        OfficerId = table.Column<Guid>(type: "uuid", nullable: false),
        AssignedByAdminId = table.Column<Guid>(type: "uuid", nullable: false),
        AssignedAt = table.Column<DateTime>(type: "timestamp with time zone", nullable: false),
        Notes = table.Column<string>(type: "character varying(1000)", maxLength: 1000, nullable: true),
        CreatedAt = table.Column<DateTime>(type: "timestamp with time zone", nullable: false),
        UpdatedAt = table.Column<DateTime>(type: "timestamp with time zone", nullable: true)
    },
    constraints: table =>
    {
        table.PrimaryKey("PK_Assignments", x => x.Id);
        table.ForeignKey(
            name: "FK_Assignments_ServiceRequests_ServiceRequestId",
            column: x => x.ServiceRequestId,
            principalTable: "ServiceRequests",
            principalColumn: "Id",
            onDelete: ReferentialAction.Cascade);
        table.ForeignKey(
            name: "FK_Assignments_Users_OfficerId",
            column: x => x.OfficerId,
            principalTable: "Users",
            principalColumn: "Id",
            onDelete: ReferentialAction.Restrict);
    });

migrationBuilder.CreateTable(
    name: "StatusUpdates",
    columns: table => new
    {
        Id = table.Column<Guid>(type: "uuid", nullable: false),
        ServiceRequestId = table.Column<Guid>(type: "uuid", nullable: false),
        OldStatus = table.Column<string>(type: "character varying(20)", maxLength: 20, nullable: false),

```

```

        NewStatus = table.Column<string>(type: "character varying(20)", maxLength: 20, nullable: false),
        ChangedById = table.Column<Guid>(type: "uuid", nullable: false),
        Comment = table.Column<string>(type: "character varying(1000)", maxLength: 1000, nullable: true),
        CreatedAt = table.Column<DateTime>(type: "timestamp with time zone", nullable: false),
        UpdatedAt = table.Column<DateTime>(type: "timestamp with time zone", nullable: true)
    },
    constraints: table =>
    {
        table.PrimaryKey("PK_StatusUpdates", x => x.Id);
        table.ForeignKey(
            name: "FK_StatusUpdates_ServiceRequests_ServiceRequestId",
            column: x => x.ServiceRequestId,
            principalTable: "ServiceRequests",
            principalColumn: "Id",
            onDelete: ReferentialAction.Cascade);
        table.ForeignKey(
            name: "FK_StatusUpdates_Users_ChangedById",
            column: x => x.ChangedById,
            principalTable: "Users",
            principalColumn: "Id",
            onDelete: ReferentialAction.Restrict);
    });

migrationBuilder.CreateIndex(
    name: "IX_Assignments_OfficerId",
    table: "Assignments",
    column: "OfficerId");

migrationBuilder.CreateIndex(
    name: "IX_Assignments_ServiceRequestId",
    table: "Assignments",
    column: "ServiceRequestId");

migrationBuilder.CreateIndex(
    name: "IX_Categories_Name",
    table: "Categories",
    column: "Name",
    unique: true);

migrationBuilder.CreateIndex(
    name: "IX_Roles_Name",
    table: "Roles",
    column: "Name",
    unique: true);

migrationBuilder.CreateIndex(
    name: "IX_ServiceRequests_CategoryId",
    table: "ServiceRequests",
    column: "CategoryId");

migrationBuilder.CreateIndex(
    name: "IX_ServiceRequests_RequesterId",
    table: "ServiceRequests",
    column: "RequesterId");

migrationBuilder.CreateIndex(
    name: "IX_StatusUpdates_ChangedById",
    table: "StatusUpdates",
    column: "ChangedById");

migrationBuilder.CreateIndex(
    name: "IX_StatusUpdates_ServiceRequestId",
    table: "StatusUpdates",
    column: "ServiceRequestId");

migrationBuilder.CreateIndex(
    name: "IX_Users_Email",
    table: "Users",
    column: "Email",
    unique: true);

migrationBuilder.CreateIndex(
    name: "IX_Users_RoleId",
    table: "Users",
    column: "RoleId");
}

/// <inheritdoc />
protected override void Down(MigrationBuilder migrationBuilder)
{
    migrationBuilder.DropTable(
        name: "Assignments");

    migrationBuilder.DropTable(
        name: "StatusUpdates");

    migrationBuilder.DropTable(
        name: "ServiceRequests");

    migrationBuilder.DropTable(
        name: "Categories");

    migrationBuilder.DropTable(
        name: "Users");
}

```

```

        migrationBuilder.DropTable(
            name: "Roles");
    }
}
}

```

UniversityMaintenance.Infrastructure.Security.JwtSettings.cs

C#

```

namespace UniversityMaintenance.Infrastructure.Security;

/// <summary>Bound from the "Jwt" configuration section.</summary>
public class JwtSettings
{
    public const string SectionName = "Jwt";

    public string Key { get; set; } = string.Empty;
    public string Issuer { get; set; } = "UniversityMaintenance";
    public string Audience { get; set; } = "UniversityMaintenance";
    public int ExpiryMinutes { get; set; } = 120;
}

```

UniversityMaintenance.Infrastructure.Security.JwtTokenService.cs

C#

```

using System.IdentityModel.Tokens.Jwt;
using System.Security.Claims;
using System.Text;
using Microsoft.Extensions.Options;
using Microsoft.IdentityModel.Tokens;
using UniversityMaintenance.Application.Common.Interfaces;
using UniversityMaintenance.Domain.Entities;

namespace UniversityMaintenance.Infrastructure.Security;

public class JwtTokenService(IOptions<JwtSettings> options) : IJwtTokenService
{
    private readonly JwtSettings _settings = options.Value;

    public (string Token, DateTime ExpiresAt) CreateToken(User user)
    {
        var expiresAt = DateTime.UtcNow.AddMinutes(_settings.ExpiryMinutes);

        var claims = new List<Claim>
        {
            new(JwtRegisteredClaimNames.Sub, user.Id.ToString()),
            new(JwtRegisteredClaimNames.Email, user.Email),
            new(ClaimTypes.NameIdentifier, user.Id.ToString()),
            new(ClaimTypes.Name, user.FullName),
            new(ClaimTypes.Role, user.Role?.Name ?? string.Empty),
            new(JwtRegisteredClaimNames.Jti, Guid.NewGuid().ToString())
        };

        var key = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(_settings.Key));
        var credentials = new SigningCredentials(key, SecurityAlgorithms.HmacSha256);

        var token = new JwtSecurityToken(
            issuer: _settings.Issuer,
            audience: _settings.Audience,
            claims: claims,
            expires: expiresAt,
            signingCredentials: credentials);

        return (new JwtSecurityTokenHandler().WriteToken(token), expiresAt);
    }
}

```

UniversityMaintenance.Infrastructure.Security.PasswordHasherService.cs

C#

```

using Microsoft.AspNetCore.Identity;
using UniversityMaintenance.Application.Common.Interfaces;
using UniversityMaintenance.Domain.Entities;

namespace UniversityMaintenance.Infrastructure.Security;

/// <summary>Wraps ASP.NET Core's PBKDF2-based <see cref="PasswordHasher{T}">.</summary>
public class PasswordHasherService : IPasswordHasher
{
    private readonly PasswordHasher<User> _hasher = new();

    public string Hash(string password) => _hasher.HashPassword(new User(), password);

    public bool Verify(string hash, string password)
    {
        var result = _hasher.VerifyHashedPassword(new User(), hash, password);
        return result is PasswordVerificationResult.Success
            or PasswordVerificationResult.SuccessRehashNeeded;
    }
}

```

UniversityMaintenance.Infrastructure.Storage.FileStorageOptions.cs

C#

```

namespace UniversityMaintenance.Infrastructure.Storage;

public class FileStorageOptions
{
    public const string SectionName = "FileStorage";

    /// <summary>Absolute path on disk where uploads are written (set by the host at startup).</summary>
    public string RootPath { get; set; } = string.Empty;

    /// <summary>Public URL prefix the saved files are served from.</summary>
    public string RequestPath { get; set; } = "/uploads";

    public long MaxBytes { get; set; } = 5 * 1024 * 1024; // 5 MB

    public string[] AllowedExtensions { get; set; } = [".jpg", ".jpeg", ".png", ".gif", ".webp"];
}

```

UniversityMaintenance.Infrastructure.Storage.FileStorageService.cs

C#

```

using Microsoft.Extensions.Options;
using UniversityMaintenance.Application.Common.Exceptions;
using UniversityMaintenance.Application.Common.Interfaces;

namespace UniversityMaintenance.Infrastructure.Storage;

/// <summary>Saves evidence images to local disk and returns a web-servable relative path.</summary>
public class FileStorageService(IOptions<FileStorageOptions> options) : IFileStorageService
{
    private readonly FileStorageOptions _options = options.Value;

    public async Task<string> SaveAsync(Stream content, string fileName, string contentType,
        long length, CancellationToken cancellationToken = default)
    {
        if (length <= 0)
            throw new BadRequestException("The uploaded file is empty.");

        if (length > _options.MaxBytes)
            throw new BadRequestException($"File exceeds the {_options.MaxBytes / (1024 * 1024)} MB limit.");

        var ext = Path.GetExtension(fileName).ToLowerInvariant();
        if (!_options.AllowedExtensions.Contains(ext))
            throw new BadRequestException("Unsupported file type. Allowed: " +
                string.Join(", ", _options.AllowedExtensions));

        Directory.CreateDirectory(_options.RootPath);

        var storedName = $"{Guid.NewGuid():N}{ext}";
        var physicalPath = Path.Combine(_options.RootPath, storedName);

        await using (var stream = new FileStream(physicalPath, FileMode.Create))
        {
            await content.CopyToAsync(stream, cancellationToken);
        }

        // e.g. "/uploads/ab12...jpg"
        return $"{_options.RequestPath.TrimEnd('/')}/{storedName}";
    }
}

```

UniversityMaintenance.Infrastructure.UniversityMaintenance.Infrastructure.csproj

XML

```

<Project Sdk="Microsoft.NET.Sdk">

  <ItemGroup>
    <ProjectReference Include="..\UniversityMaintenance.Application\UniversityMaintenance.Application.csproj" />
    <ProjectReference Include="..\UniversityMaintenance.Domain\UniversityMaintenance.Domain.csproj" />
  </ItemGroup>

  <ItemGroup>
    <PackageReference Include="Microsoft.EntityFrameworkCore.Design" Version="10.0.9">
      <IncludeAssets>runtime; build; native; contentfiles; analyzers; buildtransitive</IncludeAssets>
      <PrivateAssets>all</PrivateAssets>
    </PackageReference>
    <PackageReference Include="Microsoft.EntityFrameworkCore.Relational" Version="10.0.9" />
    <PackageReference Include="Microsoft.Extensions.Identity.Core" Version="10.0.9" />
    <PackageReference Include="Npgsql.EntityFrameworkCore.PostgreSQL" Version="10.0.2" />
    <PackageReference Include="System.IdentityModel.Tokens.Jwt" Version="8.19.1" />
  </ItemGroup>

  <PropertyGroup>
    <TargetFramework>net10.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
  </PropertyGroup>

</Project>

```


API layer

UniversityMaintenance.API/Common/CurrentUser.cs

C#

```
using System.Security.Claims;
using UniversityMaintenance.Application.Common.Interfaces;

namespace UniversityMaintenance.API.Common;

/// <summary>Reads the authenticated user's identity from the current HTTP request's claims.</summary>
public class CurrentUser(IHttpContextAccessor accessor) : ICurrentUser
{
    private ClaimsPrincipal? Principal => accessor.HttpContext?.User;

    public Guid? UserId =>
        Guid.TryParse(Principal?.FindFirstValue(ClaimTypes.NameIdentifier), out var id) ? id : null;

    public string? Email => Principal?.FindFirstValue(ClaimTypes.Email);

    public string? Role => Principal?.FindFirstValue(ClaimTypes.Role);

    public bool IsInRole(string role) => Principal?.IsInRole(role) ?? false;
}
```

UniversityMaintenance.API/Controllers/AssignmentsController.cs

C#

```
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using UniversityMaintenance.Application.DTOs;
using UniversityMaintenance.Application.Services.Interfaces;
using UniversityMaintenance.Domain.Entities;

namespace UniversityMaintenance.API.Controllers;

[ApiController]
[Authorize(Roles = RoleNames.Admin)]
[Route("api/[controller]")]
public class AssignmentsController(IAssignmentService assignments) : ControllerBase
{
    /// <summary>Assign a service request to a maintenance officer (admin only).</summary>
    [HttpPost]
    public async Task<ActionResult<AssignmentDto>> Assign(CreateAssignmentDto dto, CancellationToken ct)
        => Ok(await assignments.AssignAsync(dto, ct));
}
```

UniversityMaintenance.API/Controllers/AuthController.cs

C#

```
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using UniversityMaintenance.Application.DTOs;
using UniversityMaintenance.Application.Services.Interfaces;

namespace UniversityMaintenance.API.Controllers;

[ApiController]
[Route("api/[controller]")]
public class AuthController(IAuthService authService) : ControllerBase
{
    /// <summary>Register a new student/staff account.</summary>
    [HttpPost("register")]
    [AllowAnonymous]
    public async Task<ActionResult<AuthResponseDto>> Register(RegisterDto dto, CancellationToken ct)
        => Ok(await authService.RegisterAsync(dto, ct));

    /// <summary>Log in and receive a JWT bearer token.</summary>
    [HttpPost("login")]
    [AllowAnonymous]
    public async Task<ActionResult<AuthResponseDto>> Login(LoginDto dto, CancellationToken ct)
        => Ok(await authService.LoginAsync(dto, ct));
}
```

UniversityMaintenance.API/Controllers/CategoriesController.cs

C#

```
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using UniversityMaintenance.Application.DTOs;
using UniversityMaintenance.Application.Services.Interfaces;
using UniversityMaintenance.Domain.Entities;

namespace UniversityMaintenance.API.Controllers;

[ApiController]
[Authorize]
[Route("api/[controller]")]
public class CategoriesController(ICategoryService categories) : ControllerBase
{
    /// <summary>List all request categories (any authenticated user).</summary>
}
```

```

[HttpGet]
public async Task<ActionResult<IReadOnlyList<CategoryDto>>> GetAll(Cancellation token ct)
    => Ok(await categories.GetAllAsync(ct));

[HttpPost]
[Authorize(Roles = RoleNames.Admin)]
public async Task<ActionResult<CategoryDto>> Create(CreateCategoryDto dto, Cancellation token ct)
    => Ok(await categories.CreateAsync(dto, ct));

[HttpPut("{id:guid}")]
[Authorize(Roles = RoleNames.Admin)]
public async Task<ActionResult<CategoryDto>> Update(Guid id, CreateCategoryDto dto, Cancellation token ct)
    => Ok(await categories.UpdateAsync(id, dto, ct));

[HttpDelete("{id:guid}")]
[Authorize(Roles = RoleNames.Admin)]
public async Task<ActionResult> Delete(Guid id, Cancellation token ct)
{
    await categories.DeleteAsync(id, ct);
    return NoContent();
}
}

```

UniversityMaintenance.API/Controllers/ReportsController.cs

C#

```

using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using UniversityMaintenance.Application.DTOS;
using UniversityMaintenance.Application.Services.Interfaces;
using UniversityMaintenance.Domain.Entities;

namespace UniversityMaintenance.API.Controllers;

[ApiController]
[Authorize(Roles = RoleNames.Admin)]
[Route("api/[controller]")]
public class ReportsController(IReportService reports) : ControllerBase
{
    /// <summary>Aggregate counts for the admin dashboard (admin only).</summary>
    [HttpGet("summary")]
    public async Task<ActionResult<ReportSummaryDto>> Summary(Cancellation token ct)
        => Ok(await reports.GetSummaryAsync(ct));
}

```

UniversityMaintenance.API/Controllers/ServiceRequestsController.cs

C#

```

using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using UniversityMaintenance.Application.Common.Interfaces;
using UniversityMaintenance.Application.Common.Models;
using UniversityMaintenance.Application.DTOS;
using UniversityMaintenance.Application.Services.Interfaces;

namespace UniversityMaintenance.API.Controllers;

[ApiController]
[Authorize]
[Route("api/service-requests")]
public class ServiceRequestsController(
    IServiceRequestService requests,
    IFileStorageService fileStorage) : ControllerBase
{
    /// <summary>Paged, searchable, filterable list scoped to the caller's role.</summary>
    [HttpGet]
    public async Task<ActionResult<PagedResult<ServiceRequestDto>>> Get(
        [FromQuery] ServiceRequestQuery query, Cancellation token ct)
        => Ok(await requests.GetPagedAsync(query, ct));

    [HttpGet("{id:guid}")]
    public async Task<ActionResult<ServiceRequestDto>> GetById(Guid id, Cancellation token ct)
        => Ok(await requests.GetByIdAsync(id, ct));

    /// <summary>Status history / audit trail for a request.</summary>
    [HttpGet("{id:guid}/history")]
    public async Task<ActionResult<IReadOnlyList<StatusUpdateDto>>> GetHistory(Guid id, Cancellation token ct)
        => Ok(await requests.GetHistoryAsync(id, ct));

    /// <summary>Submit a new service request with an optional evidence image (multipart form).</summary>
    [HttpPost]
    [Consumes("multipart/form-data")]
    public async Task<ActionResult<ServiceRequestDto>> Create(
        [FromForm] CreateServiceRequestDto dto, IFormFile? evidence, Cancellation token ct)
    {
        string? evidencePath = null;
        if (evidence is { Length: > 0 })
        {
            await using var stream = evidence.OpenReadStream();
            evidencePath = await fileStorage.SaveAsync(
                stream, evidence.FileName, evidence.ContentType, evidence.Length, ct);
        }
    }
}

```

```

        var created = await requests.CreateAsync(dto, evidencePath, ct);
        return CreatedAtAction(nameof(GetById), new { id = created.Id }, created);
    }

    /// <summary>Update the status of a request (assigned officer or admin only).</summary>
    [HttpPatch("{id:guid}/status")]
    public async Task<ActionResult<ServiceRequestDto>> UpdateStatus(
        Guid id, UpdateStatusDto dto, CancellationToken ct)
        => Ok(await requests.UpdateStatusAsync(id, dto, ct));

    [HttpDelete("{id:guid}")]
    public async Task<ActionResult> Delete(Guid id, CancellationToken ct)
    {
        await requests.DeleteAsync(id, ct);
        return NoContent();
    }
}

```

UniversityMaintenance.API/Controllers/UsersController.cs

C#

```

using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using UniversityMaintenance.Application.Common.Models;
using UniversityMaintenance.Application.DTOS;
using UniversityMaintenance.Application.Services.Interfaces;
using UniversityMaintenance.Domain.Entities;

namespace UniversityMaintenance.API.Controllers;

[ApiController]
[Authorize(Roles = RoleNames.Admin)]
[Route("api/[controller]")]
public class UsersController(IUserService users) : ControllerBase
{
    /// <summary>Paged, searchable list of all users (admin only).</summary>
    [HttpGet]
    public async Task<ActionResult<PagedResult<UserDto>>> Get(
        [FromQuery] string? search, [FromQuery] int page = 1, [FromQuery] int pageSize = 10,
        CancellationToken ct = default)
        => Ok(await users.GetPagedAsync(search, page, pageSize, ct));

    /// <summary>list maintenance officers (for the assignment dropdown).</summary>
    [HttpGet("officers")]
    public async Task<ActionResult<IReadOnlyList<UserDto>>> GetOfficers(CancellationToken ct)
        => Ok(await users.GetOfficersAsync(ct));

    /// <summary>Create an Officer or Admin account (admin only).</summary>
    [HttpPost]
    public async Task<ActionResult<UserDto>> Create(CreateUserDto dto, CancellationToken ct)
        => Ok(await users.CreateAsync(dto, ct));

    [HttpDelete("{id:guid}")]
    public async Task<ActionResult> Delete(Guid id, CancellationToken ct)
    {
        await users.DeleteAsync(id, ct);
        return NoContent();
    }
}

```

UniversityMaintenance.API/Dockerfile

TXT

```

# ---- Build stage ----
FROM mcr.microsoft.com/dotnet/sdk:10.0 AS build
WORKDIR /src

# Copy csproj files first for restore-layer caching.
COPY UniversityMaintenance.Domain/UniversityMaintenance.Domain.csproj UniversityMaintenance.Domain/
COPY UniversityMaintenance.Application/UniversityMaintenance.Application.csproj UniversityMaintenance.Application/
COPY UniversityMaintenance.Infrastructure/UniversityMaintenance.Infrastructure.csproj UniversityMaintenance.Infrastructure/
COPY UniversityMaintenance.API/UniversityMaintenance.API.csproj UniversityMaintenance.API/
RUN dotnet restore UniversityMaintenance.API/UniversityMaintenance.API.csproj

# Copy the rest and publish.
COPY . .
RUN dotnet publish UniversityMaintenance.API/UniversityMaintenance.API.csproj -c Release -o /app/publish

# ---- Runtime stage ----
FROM mcr.microsoft.com/dotnet/aspnet:10.0 AS runtime
WORKDIR /app
COPY --from=build /app/publish .
# Render/Railway inject PORT at runtime; Program.cs binds to it.
ENV ASPNETCORE_ENVIRONMENT=Production
EXPOSE 8080
ENTRYPOINT ["dotnet", "UniversityMaintenance.API.dll"]

```

UniversityMaintenance.API/Filters/ValidationFilter.cs

C#

```

using FluentValidation;
using Microsoft.AspNetCore.Mvc;

```

```

using Microsoft.AspNetCore.Mvc.Filters;

namespace UniversityMaintenance.API.Filters;

/// <summary>
/// Runs any registered FluentValidation validator against action arguments and
/// short-circuits with a 400 + field errors when validation fails.
/// </summary>
public class ValidationFilter(IServiceProvider serviceProvider) : IAsyncActionFilter
{
    public async Task OnActionExecutionAsync(ActionExecutingContext context, ActionExecutionDelegate next)
    {
        foreach (var argument in context.ActionArguments.Values)
        {
            if (argument is null) continue;

            var validatorType = typeof(IValidator<>).MakeGenericType(argument.GetType());
            if (serviceProvider.GetService(validatorType) is not IValidator validator) continue;

            var validationContext = new ValidationContext<object>(argument);
            var result = await validator.ValidateAsync(validationContext);
            if (result.IsValid) continue;

            var errors = result.Errors
                .GroupBy(e => e.PropertyName)
                .ToDictionary(g => g.Key, g => g.Select(e => e.ErrorMessage).ToArray());

            context.Result = new BadRequestObjectResult(new ValidationProblemDetails(errors));
            return;
        }

        await next();
    }
}

```

UniversityMaintenance.API/Middleware/ExceptionHandlerMiddleware.cs

C#

```

using System.Text.Json;
using UniversityMaintenance.Application.Common.Exceptions;

namespace UniversityMaintenance.API.Middleware;

/// <summary>Translates thrown exceptions into consistent JSON error responses.</summary>
public class ExceptionHandlingMiddleware(RequestDelegate next, ILogger<ExceptionHandlerMiddleware> logger)
{
    public async Task InvokeAsync(HttpContext context)
    {
        try
        {
            await next(context);
        }
        catch (Exception ex)
        {
            var (status, message) = ex switch
            {
                NotFoundException => (StatusCodes.Status404NotFound, ex.Message),
                ForbiddenException => (StatusCodes.Status403Forbidden, ex.Message),
                ConflictException => (StatusCodes.Status409Conflict, ex.Message),
                BadRequestException => (StatusCodes.Status400BadRequest, ex.Message),
                _ => (StatusCodes.Status500InternalServerError, "An unexpected error occurred.")
            };

            if (status == StatusCodes.Status500InternalServerError)
                logger.LogError(ex, "Unhandled exception processing {Path}", context.Request.Path);

            context.Response.StatusCode = status;
            context.Response.ContentType = "application/json";
            var payload = JsonSerializer.Serialize(new { error = message });
            await context.Response.WriteAsync(payload);
        }
    }
}

```

UniversityMaintenance.API/Program.cs

C#

```

using System.Text;
using Microsoft.AspNetCore.Authentication.JwtBearer;
using Microsoft.Extensions.FileProviders;
using Microsoft.IdentityModel.Tokens;
using Microsoft.OpenApi.Models;
using UniversityMaintenance.API.Common;
using UniversityMaintenance.API.Filters;
using UniversityMaintenance.API.Middleware;
using UniversityMaintenance.Application;
using UniversityMaintenance.Application.Common.Interfaces;
using UniversityMaintenance.Infrastructure;
using UniversityMaintenance.Infrastructure.Persistence;
using UniversityMaintenance.Infrastructure.Security;
using UniversityMaintenance.Infrastructure.Storage;

var builder = WebApplication.CreateBuilder(args);

```

```

// Honor the PORT env var injected by cloud hosts (Render, Railway, Heroku).
var port = Environment.GetEnvironmentVariable("PORT");
if (!string.IsNullOrEmpty(port))
    builder.WebHost.UseUrls($"http://0.0.0.0:{port}");

// ---- Configuration objects ----
var jwtSettings = new JwtSettings();
builder.Configuration.GetSection(JwtSettings.SectionName).Bind(jwtSettings);

var corsOrigins = builder.Configuration.GetSection("Cors:AllowedOrigins").Get<string[]>()
    ?? ["http://localhost:5173"];

// ---- Services ----
builder.Services.AddControllers(options => options.Filters.Add<ValidationFilter>());

builder.Services.AddApplication();
builder.Services.AddInfrastructure(builder.Configuration);

builder.Services.AddHttpContextAccessor();
builder.Services.AddScoped<ICurrentUser, CurrentUser>();

// Resolve the physical uploads folder (served statically at /uploads).
var uploadsPath = Path.Combine(builder.Environment.ContentRootPath, "wwwroot", "uploads");
Directory.CreateDirectory(uploadsPath);
builder.Services.PostConfigure<FileStorageOptions>(o => o.RootPath = uploadsPath);

// ---- Authentication / Authorization ----
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(options =>
    {
        options.TokenValidationParameters = new TokenValidationParameters
        {
            ValidateIssuer = true,
            ValidateAudience = true,
            ValidateLifetime = true,
            ValidateIssuerSigningKey = true,
            ValidIssuer = jwtSettings.Issuer,
            ValidAudience = jwtSettings.Audience,
            IssuerSigningKey = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(jwtSettings.Key)),
            ClockSkew = TimeSpan.Zero
        };
    });
builder.Services.AddAuthorization();

// ---- CORS ----
builder.Services.AddCors(options => options.AddDefaultPolicy(policy =>
    policy.WithOrigins(corsOrigins).AllowAnyHeader().AllowAnyMethod()));

// ---- Swagger with JWT support ----
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen(c =>
{
    c.SwaggerDoc("v1", new OpenApiInfo
    {
        Title = "University Maintenance System API",
        Version = "v1",
        Description = "REST API for submitting, assigning and tracking maintenance service requests."
    });

    var scheme = new OpenApiSecurityScheme
    {
        Name = "Authorization",
        Type = SecuritySchemeType.Http,
        Scheme = "bearer",
        BearerFormat = "JWT",
        In = ParameterLocation.Header,
        Description = "Enter the JWT token (without the 'Bearer' prefix).",
        Reference = new OpenApiReference { Type = ReferenceType.SecurityScheme, Id = "Bearer" }
    };
    c.AddSecurityDefinition("Bearer", scheme);
    c.AddSecurityRequirement(new OpenApiSecurityRequirement { [scheme] = Array.Empty<string>() });
});

var app = builder.Build();

// ---- Migrate + seed the database on startup ----
await using (var scope = app.Services.CreateAsyncScope())
{
    var db = scope.ServiceProvider.GetRequiredService<AppDbContext>();
    var hasher = scope.ServiceProvider.GetRequiredService<IPasswordHasher>();
    var adminEmail = builder.Configuration["Seed:AdminEmail"] ?? "admin@university.edu";
    var adminPassword = builder.Configuration["Seed:AdminPassword"] ?? "Admin@123";
    await DbSeeder.SeedAsync(db, hasher, adminEmail, adminPassword);
}

// ---- HTTP pipeline ----
app.UseMiddleware<ExceptionHandlerMiddleware>();

app.UseSwagger();
app.UseSwaggerUI(c => c.SwaggerEndpoint("/swagger/v1/swagger.json", "University Maintenance API v1"));

// Serve uploaded evidence images at /uploads via an explicit provider (robust even when
// wwwroot does not exist at host-config time).
app.UseStaticFiles(new StaticFileOptions

```

```
{
    FileProvider = new PhysicalFileProvider(uploadsPath),
    RequestPath = "/uploads"
});

app.UseCors();
app.UseAuthentication();
app.UseAuthorization();

app.MapControllers();

app.Run();

// Exposed so integration tests can reference the entry point via WebApplicationFactory.
public partial class Program;
```

UniversityMaintenance.API/Properties/launchSettings.json

JSON

```
{
  "$schema": "https://json.schemastore.org/launchsettings.json",
  "profiles": {
    "http": {
      "commandName": "Project",
      "dotnetRunMessages": true,
      "launchBrowser": false,
      "applicationUrl": "http://localhost:5071",
      "environmentVariables": {
        "ASPNETCORE_ENVIRONMENT": "Development"
      }
    },
    "https": {
      "commandName": "Project",
      "dotnetRunMessages": true,
      "launchBrowser": false,
      "applicationUrl": "https://localhost:7248;http://localhost:5071",
      "environmentVariables": {
        "ASPNETCORE_ENVIRONMENT": "Development"
      }
    }
  }
}
```

UniversityMaintenance.API/UniversityMaintenance.API.csproj

XML

```
<Project Sdk="Microsoft.NET.Sdk.Web">

  <PropertyGroup>
    <TargetFramework>net10.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
  </PropertyGroup>

  <ItemGroup>
    <PackageReference Include="Microsoft.AspNetCore.Authentication.JwtBearer" Version="10.0.9" />
    <PackageReference Include="Swashbuckle.AspNetCore" Version="7.2.0" />
  </ItemGroup>

  <ItemGroup>
    <ProjectReference Include="..\UniversityMaintenance.Application\UniversityMaintenance.Application.csproj" />
    <ProjectReference Include="..\UniversityMaintenance.Infrastructure\UniversityMaintenance.Infrastructure.csproj" />
  </ItemGroup>

</Project>
```

UniversityMaintenance.API/appsettings.Development.json

JSON

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  }
}
```

UniversityMaintenance.API/appsettings.json

JSON

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Host=localhost;Port=5432;Database=university_maintenance;Username=postgres;Password=postgres"
  },
  "Jwt": {
    "Key": "REPLACE_THIS_WITH_A_LONG_RANDOM_SECRET_AT_LEAST_32_CHARS",
    "Issuer": "UniversityMaintenance",
    "Audience": "UniversityMaintenanceClient",
    "ExpiryMinutes": 120
  },
}
```

```

"Cors": {
  "AllowedOrigins": [ "http://localhost:5173" ]
},
"FileStorage": {
  "RequestPath": "/uploads",
  "MaxBytes": 5242880
},
"Seed": {
  "AdminEmail": "admin@university.edu",
  "AdminPassword": "Admin@123"
},
"Logging": {
  "LogLevel": {
    "Default": "Information",
    "Microsoft.AspNetCore": "Warning"
  }
},
"AllowedHosts": "*"
}

```

Automated tests

UniversityMaintenance.Tests/ApiFactory.cs

C#

```

using Microsoft.AspNetCore.Hosting;
using Microsoft.AspNetCore.Mvc.Testing;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.DependencyInjection;
using UniversityMaintenance.Infrastructure.Persistence;

namespace UniversityMaintenance.Tests;

/// <summary>
/// Boots the real API pipeline (auth, RBAC, controllers, validation) but swaps the
/// PostgreSQL context for an isolated in-memory database so tests need no live server.
/// </summary>
public class ApiFactory : WebApplicationFactory<Program>
{
    private readonly string _dbName = Guid.NewGuid().ToString();

    protected override void ConfigureWebHost(IWebHostBuilder builder)
    {
        builder.UseEnvironment("Testing");

        builder.ConfigureServices(services =>
        {
            // Remove every EF descriptor tied to AppDbContext, including the EF 9/10
            // IDbContextOptionsConfiguration<AppDbContext> that carries the Npgsql setup.
            var toRemove = services
                .Where(d =>
                    d.ServiceType == typeof(DbContextOptions<AppDbContext>) ||
                    d.ServiceType == typeof(AppDbContext) ||
                    (d.ServiceType.IsGenericType &&
                     d.ServiceType.GetGenericArguments().Contains(typeof(AppDbContext))))
                .ToList();
            foreach (var d in toRemove) services.Remove(d);

            services.AddDbContext<AppDbContext>(o => o.UseInMemoryDatabase(_dbName));
        });
    }
}

```

UniversityMaintenance.Tests/ApiIntegrationTests.cs

C#

```

using System.Net;
using System.Net.Http.Json;
using System.Text.Json;
using UniversityMaintenance.Application.DTOS;

namespace UniversityMaintenance.Tests;

public class ApiIntegrationTests(ApiFactory factory) : IClassFixture<ApiFactory>
{
    private readonly HttpClient _client = factory.CreateClient();

    private static readonly JsonSerializerOptions Json = new(JsonSerializerDefaults.Web);

    private async Task<string> RegisterAndGetTokenAsync(string email)
    {
        var res = await _client.PostAsJsonAsync("/api/auth/register", new RegisterDto
        {
            FirstName = "Test",
            LastName = "User",
            Email = email,
            Password = "Pass@123"
        });
        res.EnsureSuccessStatusCode();
        var body = await res.Content.ReadFromJsonAsync<AuthResponseDto>(Json);
        return body!.Token;
    }
}

```

```

    }

    [Fact]
    public async Task Register_Then_Login_Works()
    {
        var email = $"user{Guid.NewGuid():N}@x.edu";
        await RegisterAndGetTokenAsync(email);

        var login = await _client.PostAsJsonAsync("/api/auth/login",
            new LoginDto { Email = email, Password = "Pass@123" });

        Assert.Equal(HttpStatusCode.OK, login.StatusCode);
    }

    [Fact]
    public async Task Register_InvalidEmail_Returns400()
    {
        var res = await _client.PostAsJsonAsync("/api/auth/register", new RegisterDto
        {
            FirstName = "Bad",
            LastName = "Email",
            Email = "not-an-email",
            Password = "Pass@123"
        });

        Assert.Equal(HttpStatusCode.BadRequest, res.StatusCode);
    }

    [Fact]
    public async Task ProtectedEndpoint_WithoutToken_Returns401()
    {
        var res = await _client.GetAsync("/api/service-requests");
        Assert.Equal(HttpStatusCode.Unauthorized, res.StatusCode);
    }

    [Fact]
    public async Task Student_CannotAccess_AdminReports_Returns403()
    {
        var token = await RegisterAndGetTokenAsync($"stud{Guid.NewGuid():N}@x.edu");
        var req = new HttpRequestMessage(HttpMethod.Get, "/api/reports/summary");
        req.Headers.Authorization = new("Bearer", token);

        var res = await _client.SendAsync(req);

        Assert.Equal(HttpStatusCode.Forbidden, res.StatusCode);
    }

    [Fact]
    public async Task RequestList_ReturnsPagedShape()
    {
        var token = await RegisterAndGetTokenAsync($"pg{Guid.NewGuid():N}@x.edu");
        var req = new HttpRequestMessage(HttpMethod.Get, "/api/service-requests?page=1&pageSize=5");
        req.Headers.Authorization = new("Bearer", token);

        var res = await _client.SendAsync(req);
        res.EnsureSuccessStatusCode();

        using var doc = JsonDocument.Parse(await res.Content.ReadAsStringAsync());
        var root = doc.RootElement;
        Assert.True(root.TryGetProperty("items", out _));
        Assert.True(root.TryGetProperty("totalCount", out _));
        Assert.True(root.TryGetProperty("totalPages", out _));
    }
}

```

UniversityMaintenance.Tests/AuthServiceTests.cs

C#

```

using UniversityMaintenance.Application.Common.Exceptions;
using UniversityMaintenance.Application.DTOS;
using UniversityMaintenance.Application.Services;

namespace UniversityMaintenance.Tests;

public class AuthServiceTests
{
    private static AuthService Build(Infrastructure.Persistence.AppDbContext db) =>
        new(db, TestSupport.PasswordHasher(), TestSupport.JwtService());

    [Fact]
    public async Task Register_CreatesStudent_AndReturnsToken()
    {
        var db = await TestSupport.SeededContextAsync();
        var service = Build(db);

        var result = await service.RegisterAsync(new RegisterDto
        {
            FirstName = "Jane",
            LastName = "Doe",
            Email = "Jane@Student.edu",
            Password = "Pass@123"
        });

        Assert.False(string.IsNullOrWhiteSpace(result.Token));
    }
}

```



```

        Assert.Equal("Student", result.User.Role);
        Assert.Equal("jane@student.edu", result.User.Email); // normalized to lowercase
        Assert.Equal("Jane Doe", result.User.FullName);
    }

    [Fact]
    public async Task Register_DuplicateEmail_Throws()
    {
        var db = await TestSupport.SeededContextAsync();
        var service = Build(db);
        var dto = new RegisterDto { FirstName = "A", LastName = "B", Email = "dup@x.edu", Password = "Pass@123" };

        await service.RegisterAsync(dto);

        await Assert.ThrowsAsync<ConflictException>(() => service.RegisterAsync(dto));
    }

    [Fact]
    public async Task Login_WrongPassword_Throws()
    {
        var db = await TestSupport.SeededContextAsync();
        var service = Build(db);
        await service.RegisterAsync(new RegisterDto
        { FirstName = "A", LastName = "B", Email = "user@x.edu", Password = "Correct@1" });

        await Assert.ThrowsAsync<BadRequestException>(() =>
            service.LoginAsync(new LoginDto { Email = "user@x.edu", Password = "Wrong@1" }));
    }

    [Fact]
    public async Task Login_CorrectPassword_Succeeds()
    {
        var db = await TestSupport.SeededContextAsync();
        var service = Build(db);
        await service.RegisterAsync(new RegisterDto
        { FirstName = "A", LastName = "B", Email = "ok@x.edu", Password = "Correct@1" });

        var result = await service.LoginAsync(new LoginDto { Email = "ok@x.edu", Password = "Correct@1" });

        Assert.False(string.IsNullOrEmpty(result.Token));
        Assert.Equal("ok@x.edu", result.User.Email);
    }
}

```

UniversityMaintenance.Tests/ServiceRequestServiceTests.cs

C#

```

using Microsoft.EntityFrameworkCore;
using UniversityMaintenance.Application.Common.Exceptions;
using UniversityMaintenance.Application.DTOS;
using UniversityMaintenance.Application.Services;
using UniversityMaintenance.Domain.Entities;
using UniversityMaintenance.Domain.Enums;
using UniversityMaintenance.Infrastructure.Persistence;

namespace UniversityMaintenance.Tests;

public class ServiceRequestServiceTests
{
    private static async Task<User> AddUserAsync(AppDbContext db, string role, string email)
    {
        var roleId = await TestSupport.RoleIdAsync(db, role);
        var user = new User { FirstName = "T", LastName = role, Email = email, PasswordHash = "x", RoleId = roleId };
        db.Users.Add(user);
        await db.SaveChangesAsync();
        return user;
    }

    private static async Task<Guid> CategoryIdAsync(AppDbContext db) =>
        await db.Categories.Select(c => c.Id).FirstAsync();

    [Fact]
    public async Task Create_SetsSubmitted_AndWritesAuditEntry()
    {
        var db = await TestSupport.SeededContextAsync();
        var student = await AddUserAsync(db, RoleNames.Student, "s@x.edu");
        var current = new FakeCurrentUser { UserId = student.Id, Role = RoleNames.Student };
        var service = new ServiceRequestService(db, current);

        var created = await service.CreateAsync(new CreateServiceRequestDto
        {
            Title = "Leaky tap",
            Description = "Drip drip",
            Location = "Room 1",
            Priority = Priority.High,
            CategoryId = await CategoryIdAsync(db)
        }, evidenceImagePath: null);

        Assert.Equal(RequestStatus.Submitted, created.Status);
        Assert.Equal(1, await db.StatusUpdates.CountAsync(s => s.ServiceRequestId == created.Id));
    }

    [Fact]
    public async Task GetPaged_Student_SeesOnlyOwnRequests()

```

```

{
    var db = await TestSupport.SeededContextAsync();
    var catId = await CategoryIdAsync(db);
    var alice = await AddUserAsync(db, RoleNames.Student, "alice@x.edu");
    var bob = await AddUserAsync(db, RoleNames.Student, "bob@x.edu");

    db.ServiceRequests.Add(new ServiceRequest
    { Title = "A", Description = "d", Location = "1", CategoryId = catId, RequesterId = alice.Id });
    db.ServiceRequests.Add(new ServiceRequest
    { Title = "B", Description = "d", Location = "1", CategoryId = catId, RequesterId = bob.Id });
    await db.SaveChangesAsync();

    var service = new ServiceRequestService(db,
        new FakeCurrentUser { UserId = alice.Id, Role = RoleNames.Student });

    var page = await service.GetPagedAsync(new ServiceRequestQuery());

    Assert.Single(page.Items);
    Assert.Equal("A", page.Items[0].Title);
}

[Fact]
public async Task UpdateStatus_ByUnrelatedStudent_Throws()
{
    var db = await TestSupport.SeededContextAsync();
    var catId = await CategoryIdAsync(db);
    var owner = await AddUserAsync(db, RoleNames.Student, "owner@x.edu");
    var other = await AddUserAsync(db, RoleNames.Student, "other@x.edu");

    var req = new ServiceRequest
    { Title = "A", Description = "d", Location = "1", CategoryId = catId, RequesterId = owner.Id };
    db.ServiceRequests.Add(req);
    await db.SaveChangesAsync();

    var service = new ServiceRequestService(db,
        new FakeCurrentUser { UserId = other.Id, Role = RoleNames.Student });

    await Assert.ThrowsAsync<ForbiddenException>(() =>
        service.UpdateStatusAsync(req.Id, new UpdateStatusDto { Status = RequestStatus.Completed }));
}

[Fact]
public async Task UpdateStatus_ByAdmin_ChangesStatus_AndLogsHistory()
{
    var db = await TestSupport.SeededContextAsync();
    var catId = await CategoryIdAsync(db);
    var student = await AddUserAsync(db, RoleNames.Student, "st@x.edu");
    var admin = await AddUserAsync(db, RoleNames.Admin, "ad@x.edu");

    var req = new ServiceRequest
    { Title = "A", Description = "d", Location = "1", CategoryId = catId, RequesterId = student.Id };
    db.ServiceRequests.Add(req);
    await db.SaveChangesAsync();

    var service = new ServiceRequestService(db,
        new FakeCurrentUser { UserId = admin.Id, Role = RoleNames.Admin });

    var updated = await service.UpdateStatusAsync(req.Id,
        new UpdateStatusDto { Status = RequestStatus.InProgress, Comment = "Working" });

    Assert.Equal(RequestStatus.InProgress, updated.Status);
    var history = await service.GetHistoryAsync(req.Id);
    Assert.Contains(history, h => h.NewStatus == RequestStatus.InProgress && h.Comment == "Working");
}
}

```

UniversityMaintenance.Tests/TestSupport.cs

C#

```

using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Options;
using UniversityMaintenance.Application.Common.Interfaces;
using UniversityMaintenance.Domain.Entities;
using UniversityMaintenance.Infrastructure.Persistence;
using UniversityMaintenance.Infrastructure.Security;

namespace UniversityMaintenance.Tests;

/// <summary>Shared helpers for building an isolated in-memory context and real security services.</summary>
public static class TestSupport
{
    public static AppDbContext NewContext()
    {
        var options = new DbContextOptionsBuilder<AppDbContext>()
            .UseInMemoryDatabase(databaseName: Guid.NewGuid().ToString())
            .Options;
        return new AppDbContext(options);
    }

    public static async Task<AppDbContext> SeededContextAsync()
    {
        var db = NewContext();
        db.Roles.AddRange(
            new Role { Name = RoleNames.Student },

```

```

        new Role { Name = RoleNames.Officer },
        new Role { Name = RoleNames.Admin });
db.Categories.Add(new RequestCategory { Name = "Electricity", Description = "Power" });
await db.SaveChangesAsync();
return db;
}

public static IPasswordHasher PasswordHasher() => new PasswordHasherService();

public static IJwtTokenService JwtService() =>
    new JwtTokenService(Options.Create(new JwtSettings
    {
        Key = "test-signing-key-that-is-definitely-long-enough-1234567890",
        Issuer = "test",
        Audience = "test",
        ExpiryMinutes = 60
    })));

public static Task<Guid> RoleIdAsync(AppDbContext db, string name) =>
    db.Roles.Where(r => r.Name == name).Select(r => r.Id).FirstAsync();
}

/// <summary>Test double for the authenticated-user accessor.</summary>
public class FakeCurrentUser : ICurrentUser
{
    public Guid? UserId { get; set; }
    public string? Email { get; set; }
    public string? Role { get; set; }
    public bool IsInRole(string role) => Role == role;
}

```

UniversityMaintenance.Tests/UniversityMaintenance.Tests.csproj

XML

```

<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <TargetFramework>net10.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
    <IsPackable>false</IsPackable>
  </PropertyGroup>

  <ItemGroup>
    <PackageReference Include="coverlet.collector" Version="6.0.4" />
    <PackageReference Include="Microsoft.AspNetCore.Mvc.Testing" Version="10.0.9" />
    <PackageReference Include="Microsoft.EntityFrameworkCore.InMemory" Version="10.0.9" />
    <PackageReference Include="Microsoft.EntityFrameworkCore.Sqlite" Version="10.0.9" />
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.14.1" />
    <PackageReference Include="xunit" Version="2.9.3" />
    <PackageReference Include="xunit.runner.visualstudio" Version="3.1.4" />
  </ItemGroup>

  <ItemGroup>
    <Using Include="Xunit" />
  </ItemGroup>

  <ItemGroup>
    <ProjectReference Include="..\UniversityMaintenance.API\UniversityMaintenance.API.csproj" />
  </ItemGroup>

</Project>

```

Frontend (React + TypeScript)

UniversityMaintenance.Web/src/App.tsx

TSX

```

import { Navigate, Route, Routes } from 'react-router-dom';
import { ProtectedRoute } from '../auth/ProtectedRoute';
import { Layout } from '../components/Layout';
import { LoginPage } from '../pages/LoginPage';
import { RegisterPage } from '../pages/RegisterPage';
import { DashboardPage } from '../pages/DashboardPage';
import { RequestsListPage } from '../pages/RequestsListPage';
import { SubmitRequestPage } from '../pages/SubmitRequestPage';
import { RequestDetailPage } from '../pages/RequestDetailPage';
import { UsersPage } from '../pages/admin/UsersPage';
import { CategoriesPage } from '../pages/admin/CategoriesPage';
import { ReportsPage } from '../pages/admin/ReportsPage';
import { Roles } from '../types';

export default function App() {
    return (
        <Routes>
            <Route path="/login" element={<LoginPage />} />
            <Route path="/register" element={<RegisterPage />} />

            <Route element={<ProtectedRoute />>
                <Route element={<Layout />>
                    <Route path="/dashboard" element={<DashboardPage />} />

```

```

    <Route path="/requests" element={<RequestsListPage />} />
    <Route path="/requests/:id" element={<RequestDetailPage />} />

    <Route element={<ProtectedRoute roles={[Roles.Student]} />>
      <Route path="/requests/new" element={<SubmitRequestPage />} />
    </Route>

    <Route element={<ProtectedRoute roles={[Roles.Admin]} />>
      <Route path="/admin/users" element={<UsersPage />} />
      <Route path="/admin/categories" element={<CategoriesPage />} />
      <Route path="/admin/reports" element={<ReportsPage />} />
    </Route>
  </Route>
</Route>

  <Route path="/" element={<Navigate to="/dashboard" replace />} />
  <Route path="*" element={<Navigate to="/dashboard" replace />} />
</Routes>
);
}

```

UniversityMaintenance.Web/src/api/client.ts

TS

```

import axios from 'axios';

const baseURL = import.meta.env.VITE_API_BASE_URL ?? 'http://localhost:5071';

export const API_ORIGIN = baseURL;

export const api = axios.create({
  baseURL: `${baseURL}/api`,
});

const TOKEN_KEY = 'ums_token';

export const tokenStore = {
  get: () => localStorage.getItem(TOKEN_KEY),
  set: (token: string) => localStorage.setItem(TOKEN_KEY, token),
  clear: () => localStorage.removeItem(TOKEN_KEY),
};

// Attach the JWT to every request.
api.interceptors.request.use((config) => {
  const token = tokenStore.get();
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

// On 401, drop the token and bounce to login.
api.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response?.status === 401) {
      tokenStore.clear();
      if (!window.location.pathname.startsWith('/login')) {
        window.location.href = '/login';
      }
    }
    return Promise.reject(error);
  },
);

/** Pulls a human-readable message out of an Axios error for toast display. */
export function errorMessage(error: unknown, fallback = 'Something went wrong.'): string {
  if (axios.isAxiosError(error)) {
    const data = error.response?.data as
    | { error?: string; title?: string; errors?: Record<string, string[]> }
    | undefined;
    if (data?.errors) {
      const first = Object.values(data.errors)[0];
      if (first?.length) return first[0];
    }
    return data?.error ?? data?.title ?? error.message ?? fallback;
  }
  return fallback;
}

/** Turns a stored relative image path (/uploads/..) into an absolute URL. */
export function imageUrl(path?: string | null): string | null {
  if (!path) return null;
  return path.startsWith('http') ? path : `${API_ORIGIN}${path}`;
}

```

UniversityMaintenance.Web/src/api/endpoints.ts

TS

```

import { api } from './client';
import type {
  AssignmentDto,
  AuthResponse,
  CategoryDto,
  PagedResult,

```

```

    ReportSummary,
    ServiceRequestDto,
    StatusUpdateDto,
    UserDto,
  } from '../types';

// ---- Auth ----
export const authApi = {
  login: (email: string, password: string) =>
    api.post<AuthResponse>('/auth/login', { email, password }).then((r) => r.data),
  register: (firstName: string, lastName: string, email: string, password: string) =>
    api.post<AuthResponse>('/auth/register', { firstName, lastName, email, password }).then((r) => r.data),
};

// ---- Categories ----
export const categoriesApi = {
  getAll: () => api.get<CategoryDto[]>('/categories').then((r) => r.data),
  create: (name: string, description?: string) =>
    api.post<CategoryDto>('/categories', { name, description }).then((r) => r.data),
  update: (id: string, name: string, description?: string) =>
    api.put<CategoryDto>(`/categories/${id}`, { name, description }).then((r) => r.data),
  remove: (id: string) => api.delete(`/categories/${id}`).then((r) => r.data),
};

// ---- Service requests ----
export interface RequestQuery {
  search?: string;
  status?: number;
  categoryId?: string;
  page?: number;
  pageSize?: number;
}

export const requestsApi = {
  getPaged: (query: RequestQuery) =>
    api
      .get<PagedResult<ServiceRequestDto>>('/service-requests', { params: query })
      .then((r) => r.data),
  getById: (id: string) =>
    api.get<ServiceRequestDto>(`/service-requests/${id}`).then((r) => r.data),
  getHistory: (id: string) =>
    api.get<StatusUpdateDto[]>(`/service-requests/${id}/history`).then((r) => r.data),
  create: (form: FormData) =>
    api
      .post<ServiceRequestDto>('/service-requests', form, {
        headers: { 'Content-Type': 'multipart/form-data' },
      })
      .then((r) => r.data),
  updateStatus: (id: string, status: number, comment?: string) =>
    api
      .patch<ServiceRequestDto>(`/service-requests/${id}/status`, { status, comment })
      .then((r) => r.data),
  remove: (id: string) => api.delete(`/service-requests/${id}`).then((r) => r.data),
};

// ---- Assignments ----
export const assignmentsApi = {
  assign: (serviceRequestId: string, officerId: string, notes?: string) =>
    api
      .post<AssignmentDto>('/assignments', { serviceRequestId, officerId, notes })
      .then((r) => r.data),
};

// ---- Users (admin) ----
export const usersApi = {
  getPaged: (search: string, page: number, pageSize: number) =>
    api
      .get<PagedResult<UserDto>>('/users', { params: { search, page, pageSize } })
      .then((r) => r.data),
  getOfficers: () => api.get<UserDto[]>('/users/officers').then((r) => r.data),
  create: (payload: {
    firstName: string;
    lastName: string;
    email: string;
    password: string;
    role: string;
  }) => api.post<UserDto>('/users', payload).then((r) => r.data),
  remove: (id: string) => api.delete(`/users/${id}`).then((r) => r.data),
};

// ---- Reports (admin) ----
export const reportsApi = {
  summary: () => api.get<ReportSummary>('/reports/summary').then((r) => r.data),
};

```

UniversityMaintenance.Web/src/auth/AuthContext.tsx

TSX

```

import { createContext, useCallback, useContext, useMemo, useState, type ReactNode } from 'react';
import { authApi } from '../../api/endpoints';
import { tokenStore } from '../../api/client';
import type { UserDto } from '../../types';

interface AuthState {

```

```

user: UserDto | null;
isAuthenticated: boolean;
login: (email: string, password: string) => Promise<UserDto>;
register: (firstName: string, lastName: string, email: string, password: string) => Promise<UserDto>;
logout: () => void;
hasRole: (...roles: string[]) => boolean;
}

const USER_KEY = 'ums_user';

const AuthContext = createContext<AuthState | undefined>(undefined);

function loadStoredUser(): UserDto | null {
  const raw = localStorage.getItem(USER_KEY);
  if (!raw || !tokenStore.get()) return null;
  try {
    return JSON.parse(raw) as UserDto;
  } catch {
    return null;
  }
}

export function AuthProvider({ children }: { children: ReactNode }) {
  const [user, setUser] = useState<UserDto | null>(loadStoredUser);

  const persist = useCallback((token: string, u: UserDto) => {
    tokenStore.set(token);
    localStorage.setItem(USER_KEY, JSON.stringify(u));
    setUser(u);
  }, []);

  const login = useCallback(
    async (email: string, password: string) => {
      const res = await authApi.login(email, password);
      persist(res.token, res.user);
      return res.user;
    },
    [persist],
  );

  const register = useCallback(
    async (firstName: string, lastName: string, email: string, password: string) => {
      const res = await authApi.register(firstName, lastName, email, password);
      persist(res.token, res.user);
      return res.user;
    },
    [persist],
  );

  const logout = useCallback(() => {
    tokenStore.clear();
    localStorage.removeItem(USER_KEY);
    setUser(null);
  }, []);

  const hasRole = useCallback(
    (...roles: string[]) => (user ? roles.includes(user.role) : false),
    [user],
  );

  const value = useMemo<AuthState>(
    () => ({ user, isAuthenticated: !!user, login, register, logout, hasRole }),
    [user, login, register, logout, hasRole],
  );

  return <AuthContext.Provider value={value}>{children}</AuthContext.Provider>;
}

// eslint-disable-next-line react-refresh/only-export-components
export function useAuth(): AuthState {
  const ctx = useContext(AuthContext);
  if (!ctx) throw new Error('useAuth must be used within AuthProvider');
  return ctx;
}

```

UniversityMaintenance.Web/src/auth/ProtectedRoute.tsx

TSX

```

import { Navigate, Outlet, useLocation } from 'react-router-dom';
import { useAuth } from './AuthContext';

/** Guards routes: requires authentication, and optionally one of the given roles. */
export function ProtectedRoute({ roles }: { roles?: string[] }) {
  const { isAuthenticated, user } = useAuth();
  const location = useLocation();

  if (!isAuthenticated) {
    return <Navigate to="/login" state={{ from: location }} replace />;
  }

  if (roles && user && !roles.includes(user.role)) {
    return <Navigate to="/dashboard" replace />;
  }
}

```

```

    return <Outlet />;
  }
}

```

UniversityMaintenance.Web/src/components/Layout.tsx

TSX

```

import { NavLink, Outlet, useNavigate } from 'react-router-dom';
import { useAuth } from '../auth/AuthContext';
import { Roles } from '../types';

export function Layout() {
  const { user, logout, hasRole } = useAuth();
  const navigate = useNavigate();

  const handleLogout = () => {
    logout();
    navigate('/login');
  };

  return (
    <div className="app-shell">
      <aside className="sidebar">
        <div className="brand">
          <span className="brand-mark">UM</span>
          <div>
            <div className="brand-title">UniMaintain</div>
            <div className="brand-sub">Service Desk</div>
          </div>
        </div>

        <nav className="nav">
          <NavLink to="/dashboard" className="nav-link">
            Dashboard
          </NavLink>

          <NavLink to="/requests" className="nav-link">
            {hasRole(Roles.Student) ? 'My Requests' : hasRole(Roles.Officer) ? 'Assigned Jobs' : 'All Requests'}
          </NavLink>

          {hasRole(Roles.Student) && (
            <NavLink to="/requests/new" className="nav-link">
              Submit Request
            </NavLink>
          )}

          {hasRole(Roles.Admin) && (
            <>
              <div className="nav-heading">Administration</div>
              <NavLink to="/admin/users" className="nav-link">
                Users
              </NavLink>
              <NavLink to="/admin/categories" className="nav-link">
                Categories
              </NavLink>
              <NavLink to="/admin/reports" className="nav-link">
                Reports
              </NavLink>
            </>
          )}
        </nav>

        <div className="sidebar-footer">
          <div className="user-chip">
            <div className="user-avatar">{user?.firstName?.[0] ?? ''}</div>
            <div className="user-meta">
              <div className="user-name">{user?.fullName}</div>
              <div className="user-role">{user?.role}</div>
            </div>
          </div>
          <button className="btn btn-ghost btn-block" onClick={handleLogout}>
            Sign out
          </button>
        </div>
      </aside>

      <main className="content">
        <Outlet />
      </main>
    </div>
  );
}

```

UniversityMaintenance.Web/src/components/Pagination.test.tsx

TSX

```

import { render, screen } from '@testing-library/react';
import userEvent from '@testing-library/user-event';
import { describe, expect, it, vi } from 'vitest';
import { Pagination } from './Pagination';

describe('Pagination', () => {
  it('renders nothing when there are no results', () => {
    const { container } = render(

```

```

    <Pagination page={1} totalPages={0} totalCount={0} onPage={() => {}} />,
  );
  expect(container.firstChild).toBeNull();
});

it('disables Prev on the first page and advances on Next', async () => {
  const onPage = vi.fn();
  render(<Pagination page={1} totalPages={3} totalCount={24} onPage={onPage} />);

  expect(screen.getByRole('button', { name: /prev/i })).toBeDisabled();

  await userEvent.click(screen.getByRole('button', { name: /next/i }));
  expect(onPage).toHaveBeenCalledWith(2);
});
});

```

UniversityMaintenance.Web/src/components/Pagination.tsx

TSX

```

interface Props {
  page: number;
  totalPages: number;
  totalCount: number;
  onPage: (page: number) => void;
}

export function Pagination({ page, totalPages, totalCount, onPage }: Props) {
  if (totalCount === 0) return null;
  return (
    <div className="pagination">
      <button className="btn btn-ghost" disabled={page <= 1} onClick={() => onPage(page - 1)}>
        < Prev
      </button>
      <span className="pagination-info">
        Page {page} of {Math.max(totalPages, 1)} · {totalCount} total
      </span>
      <button
        className="btn btn-ghost"
        disabled={page >= totalPages}
        onClick={() => onPage(page + 1)}
      >
        Next >
      </button>
    </div>
  );
}

```

UniversityMaintenance.Web/src/components/StatusBadge.test.tsx

TSX

```

import { render, screen } from '@testing-library/react';
import { describe, expect, it } from 'vitest';
import { PriorityBadge, StatusBadge } from './StatusBadge';
import { Priority, RequestStatus } from '../types';

describe('StatusBadge', () => {
  it('renders the readable status label', () => {
    render(<StatusBadge status={RequestStatus.InProgress} />);
    expect(screen.getByText('In Progress')).toBeInTheDocument();
  });

  it('renders Completed with the completed style', () => {
    const { container } = render(<StatusBadge status={RequestStatus.Completed} />);
    expect(container.querySelector('.badge-completed')).toBeTruthy();
  });
});

describe('PriorityBadge', () => {
  it('renders the priority label', () => {
    render(<PriorityBadge priority={Priority.High} />);
    expect(screen.getByText('High')).toBeInTheDocument();
  });
});

```

UniversityMaintenance.Web/src/components/StatusBadge.tsx

TSX

```

import { PRIORITY_LABELS, Priority, RequestStatus, STATUS_LABELS } from '../types';

const statusClass: Record<number, string> = {
  [RequestStatus.Submitted]: 'badge-submitted',
  [RequestStatus.Assigned]: 'badge-assigned',
  [RequestStatus.InProgress]: 'badge-progress',
  [RequestStatus.Completed]: 'badge-completed',
  [RequestStatus.Rejected]: 'badge-rejected',
};

const priorityClass: Record<number, string> = {
  [Priority.Low]: 'badge-low',
  [Priority.Medium]: 'badge-medium',
  [Priority.High]: 'badge-high',
};

```



```

export function StatusBadge({ status }: { status: RequestStatus }) {
  return <span className={`badge ${statusClass[status]}`}>{STATUS_LABELS[status]}</span>;
}

export function PriorityBadge({ priority }: { priority: Priority }) {
  return <span className={`badge ${priorityClass[priority]}`}>{PRIORITY_LABELS[priority]}</span>;
}

```

UniversityMaintenance.Web/src/components/Toast.tsx

TSX

```

import { createContext, useCallback, useContext, useState, type ReactNode } from 'react';

type ToastKind = 'success' | 'error' | 'info';
interface Toast {
  id: number;
  kind: ToastKind;
  message: string;
}

interface ToastApi {
  success: (message: string) => void;
  error: (message: string) => void;
  info: (message: string) => void;
}

const ToastContext = createContext<ToastApi | undefined>(undefined);

let nextId = 1;

export function ToastProvider({ children }: { children: ReactNode }) {
  const [toasts, setToasts] = useState<Toast[]>([]);

  const push = useCallback((kind: ToastKind, message: string) => {
    const id = nextId++;
    setToasts((t) => [...t, { id, kind, message }]);
    setTimeout(() => setToasts((t) => t.filter((x) => x.id !== id)), 4000);
  }, []);

  const api: ToastApi = {
    success: (m) => push('success', m),
    error: (m) => push('error', m),
    info: (m) => push('info', m),
  };

  return (
    <ToastContext.Provider value={api}>
      {children}
      <div className="toast-stack">
        {toasts.map((t) => (
          <div key={t.id} className={`toast toast-${t.kind}`}>
            {t.message}
          </div>
        ))}
      </div>
    </ToastContext.Provider>
  );
}

// eslint-disable-next-line react-refresh/only-export-components
export function useToast(): ToastApi {
  const ctx = useContext(ToastContext);
  if (!ctx) throw new Error('useToast must be used within ToastProvider');
  return ctx;
}

```

UniversityMaintenance.Web/src/index.css

CSS

```

:root {
  --bg: #f5f6fa;
  --surface: #ffffff;
  --surface-2: #f8fafc;
  --border: #e5e7eb;
  --text: #1f2937;
  --text-muted: #6b7280;
  --primary: #4f46e5;
  --primary-dark: #4338ca;
  --secondary: #0ea5e9;
  --danger: #dc2626;
  --shadow-sm: 0 1px 2px rgba(16, 24, 40, 0.05);
  --shadow: 0 4px 12px rgba(16, 24, 40, 0.08);
  --radius: 12px;
  --sans: system-ui, 'Segoe UI', Roboto, Helvetica, Arial, sans-serif;
}

* {
  box-sizing: border-box;
}

html,
body,

```

```

#root {
  height: 100%;
  margin: 0;
}

body {
  font-family: var(--sans);
  background: var(--bg);
  color: var(--text);
  font-size: 14px;
  line-height: 1.5;
}

h1 {
  font-size: 1.6rem;
  margin: 0;
}
h2 {
  font-size: 1.15rem;
  margin: 0;
}

a {
  color: var(--primary);
  text-decoration: none;
}
.link {
  color: var(--primary);
  background: none;
  border: none;
  cursor: pointer;
  font: inherit;
  padding: 0;
}
.link:hover {
  text-decoration: underline;
}
.link.danger {
  color: var(--danger);
}
.muted {
  color: var(--text-muted);
}
.small {
  font-size: 0.8rem;
}
.strong {
  font-weight: 600;
}

/* ---- Buttons ---- */
.btn {
  display: inline-flex;
  align-items: center;
  justify-content: center;
  gap: 6px;
  border: 1px solid transparent;
  border-radius: 8px;
  padding: 9px 16px;
  font: inherit;
  font-weight: 600;
  cursor: pointer;
  transition: background 0.15s, box-shadow 0.15s, opacity 0.15s;
}
.btn.disabled {
  opacity: 0.55;
  cursor: not-allowed;
}
.btn-primary {
  background: var(--primary);
  color: #fff;
}
.btn-primary:hover:not(:disabled) {
  background: var(--primary-dark);
}
.btn-secondary {
  background: var(--secondary);
  color: #fff;
}
.btn-ghost {
  background: var(--surface);
  border-color: var(--border);
  color: var(--text);
}
.btn-ghost:hover:not(:disabled) {
  background: var(--surface-2);
}
.btn-block {
  width: 100%;
}

/* ---- Auth pages ---- */
.auth-page {
  min-height: 100%;
  display: grid;

```

```

    place-items: center;
    padding: 24px;
    background: linear-gradient(135deg, #4f46e5 0%, #0ea5e9 100%);
  }
  .auth-card {
    width: 100%;
    max-width: 420px;
    background: var(--surface);
    border-radius: var(--radius);
    box-shadow: var(--shadow);
    padding: 32px;
  }
  .auth-brand {
    text-align: center;
    margin-bottom: 20px;
  }
  .auth-brand h1 {
    font-size: 1.4rem;
    margin: 12px 0 4px;
  }
  .auth-alt {
    text-align: center;
    margin: 16px 0 0;
    color: var(--text-muted);
  }
  .brand-mark {
    display: inline-grid;
    place-items: center;
    width: 44px;
    height: 44px;
    border-radius: 12px;
    background: var(--primary);
    color: #fff;
    font-weight: 800;
    letter-spacing: 0.5px;
  }

  /* ---- Forms ---- */
  .form {
    display: flex;
    flex-direction: column;
    gap: 14px;
  }
  .field {
    display: flex;
    flex-direction: column;
    gap: 6px;
  }
  .field > span {
    font-weight: 600;
    font-size: 0.85rem;
  }
  input,
  select,
  textarea {
    font: inherit;
    padding: 9px 11px;
    border: 1px solid var(--border);
    border-radius: 8px;
    background: var(--surface);
    color: var(--text);
    width: 100%;
  }
  input:focus,
  select:focus,
  textarea:focus {
    outline: none;
    border-color: var(--primary);
    box-shadow: 0 0 0 3px rgba(79, 70, 229, 0.15);
  }
  textarea {
    resize: vertical;
  }
  .err {
    color: var(--danger);
    font-size: 0.8rem;
  }
  .grid-2 {
    display: grid;
    grid-template-columns: 1fr 1fr;
    gap: 14px;
  }
  .form-actions {
    display: flex;
    justify-content: flex-end;
    gap: 10px;
  }

  /* ---- App shell ---- */
  .app-shell {
    display: grid;
    grid-template-columns: 250px 1fr;
    min-height: 100vh;
  }

```

```

.sidebar {
  background: #111827;
  color: #d1d5db;
  display: flex;
  flex-direction: column;
  padding: 20px 14px;
  position: sticky;
  top: 0;
  height: 100vh;
}
.brand {
  display: flex;
  align-items: center;
  gap: 12px;
  padding: 4px 8px 20px;
}
.brand-title {
  font-weight: 700;
  color: #fff;
}
.brand-sub {
  font-size: 0.75rem;
  color: #9ca3af;
}
.nav {
  display: flex;
  flex-direction: column;
  gap: 2px;
  flex: 1;
}
.nav-heading {
  font-size: 0.7rem;
  text-transform: uppercase;
  letter-spacing: 0.08em;
  color: #6b7280;
  margin: 16px 8px 6px;
}
.nav-link {
  color: #d1d5db;
  padding: 9px 12px;
  border-radius: 8px;
  font-weight: 500;
}
.nav-link:hover {
  background: rgba(255, 255, 255, 0.06);
  text-decoration: none;
}
.nav-link.active {
  background: var(--primary);
  color: #fff;
}
.sidebar-footer {
  border-top: 1px solid rgba(255, 255, 255, 0.08);
  padding-top: 14px;
  display: flex;
  flex-direction: column;
  gap: 10px;
}
.user-chip {
  display: flex;
  align-items: center;
  gap: 10px;
}
.user-avatar {
  width: 36px;
  height: 36px;
  border-radius: 50%;
  background: var(--secondary);
  color: #fff;
  display: grid;
  place-items: center;
  font-weight: 700;
}
.user-name {
  color: #fff;
  font-weight: 600;
  font-size: 0.85rem;
}
.user-role {
  font-size: 0.75rem;
  color: #9ca3af;
}
.content {
  padding: 28px 32px;
  overflow-x: auto;
}

/* ---- Pages ---- */
.page {
  max-width: 1100px;
  margin: 0 auto;
  display: flex;
  flex-direction: column;
  gap: 20px;
}

```

```

}
.page-narrow {
  max-width: 640px;
}
.page-head {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 16px;
}
.back {
  align-self: flex-start;
}

/* ---- Cards ---- */
.card {
  background: var(--surface);
  border: 1px solid var(--border);
  border-radius: var(--radius);
  box-shadow: var(--shadow-sm);
  padding: 20px;
}
.card-title {
  margin: 0 0 14px;
}
.card-head {
  display: flex;
  align-items: center;
  justify-content: space-between;
  margin-bottom: 12px;
}
.empty {
  text-align: center;
  padding: 24px;
}

/* ---- Stat cards ---- */
.stat-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(180px, 1fr));
  gap: 16px;
}
.stat-card {
  background: var(--surface);
  border: 1px solid var(--border);
  border-radius: var(--radius);
  padding: 20px;
  box-shadow: var(--shadow-sm);
  border-left: 4px solid var(--primary);
}
.stat-value {
  font-size: 2rem;
  font-weight: 800;
}
.stat-label {
  color: var(--text-muted);
  font-weight: 500;
}
.accent-blue {
  border-left-color: #3b82f6;
}
.accent-violet {
  border-left-color: #8b5cf6;
}
.accent-green {
  border-left-color: #10b981;
}
.accent-amber {
  border-left-color: #f59e0b;
}

/* ---- Tables ---- */
.filters {
  display: flex;
  gap: 12px;
  flex-wrap: wrap;
  align-items: center;
}
.filter-search {
  flex: 1;
  min-width: 220px;
}
.filters select {
  width: auto;
}
.table-wrap {
  overflow-x: auto;
}
.table {
  width: 100%;
  border-collapse: collapse;
}
.table th,
.table td {
  text-align: left;

```

```

padding: 11px 12px;
border-bottom: 1px solid var(--border);
white-space: nowrap;
}
.table th {
font-size: 0.75rem;
text-transform: uppercase;
letter-spacing: 0.04em;
color: var(--text-muted);
}
.table tbody tr:hover {
background: var(--surface-2);
}

/* ---- Badges ---- */
.badge {
display: inline-block;
padding: 3px 10px;
border-radius: 999px;
font-size: 0.75rem;
font-weight: 600;
}
.badge-submitted {
background: #e0e7ff;
color: #3730a3;
}
.badge-assigned {
background: #cffffe;
color: #0e7490;
}
.badge-progress {
background: #fef3c7;
color: #92400e;
}
.badge-completed {
background: #d1fae5;
color: #065f46;
}
.badge-rejected {
background: #fee2e2;
color: #991b1b;
}
.badge-low {
background: #f1f5f9;
color: #475569;
}
.badge-medium {
background: #fef9c3;
color: #854d0e;
}
.badge-high {
background: #fee2e2;
color: #991b1b;
}
.badge-role {
background: #ede9fe;
color: #5b21b6;
}

/* ---- Pagination ---- */
.pagination {
display: flex;
align-items: center;
justify-content: space-between;
gap: 12px;
margin-top: 14px;
}
.pagination-info {
color: var(--text-muted);
font-size: 0.85rem;
}

/* ---- Recent / bars ---- */
.recent-list {
list-style: none;
margin: 0;
padding: 0;
display: flex;
flex-direction: column;
}
.recent-item {
display: flex;
align-items: center;
justify-content: space-between;
padding: 12px 6px;
border-bottom: 1px solid var(--border);
color: inherit;
}
.recent-item:hover {
text-decoration: none;
background: var(--surface-2);
}
.recent-title {
font-weight: 600;
}

```

```

.bar-list {
  display: flex;
  flex-direction: column;
  gap: 12px;
}
.bar-row {
  display: grid;
  grid-template-columns: 120px 1fr 40px;
  align-items: center;
  gap: 12px;
}
.bar-label {
  font-size: 0.85rem;
}
.bar-track {
  background: var(--surface-2);
  border-radius: 999px;
  height: 10px;
  overflow: hidden;
}
.bar-fill {
  background: var(--primary);
  height: 100%;
  border-radius: 999px;
  transition: width 0.3s;
}
.bar-value {
  text-align: right;
  font-weight: 600;
}
.report-cols {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(280px, 1fr));
  gap: 16px;
}

/* ---- Detail page ---- */
.detail-grid {
  display: grid;
  grid-template-columns: 1fr 320px;
  gap: 20px;
  align-items: start;
}
.detail-main {
  display: flex;
  flex-direction: column;
  gap: 20px;
}
.detail-side {
  display: flex;
  flex-direction: column;
  gap: 20px;
}
.detail-head {
  display: flex;
  justify-content: space-between;
  gap: 16px;
  margin-bottom: 12px;
}
.badge-col {
  display: flex;
  flex-direction: column;
  gap: 6px;
  align-items: flex-end;
}
.detail-desc {
  white-space: pre-wrap;
}
.detail-meta {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 12px;
  margin: 16px 0 0;
}
.detail-meta dt {
  font-size: 0.75rem;
  text-transform: uppercase;
  color: var(--text-muted);
}
.detail-meta dd {
  margin: 2px 0 0;
  font-weight: 600;
}
.evidence {
  margin-top: 16px;
  display: flex;
  flex-direction: column;
  gap: 6px;
}
.evidence img {
  max-width: 260px;
  border-radius: 8px;
  border: 1px solid var(--border);
}

```

```

/* ---- Timeline ---- */
.timeline {
  list-style: none;
  margin: 0;
  padding: 0;
  display: flex;
  flex-direction: column;
  gap: 18px;
}
.timeline li {
  display: grid;
  grid-template-columns: 16px 1fr;
  gap: 12px;
  position: relative;
}
.timeline li:not(:last-child)::before {
  content: '';
  position: absolute;
  left: 7px;
  top: 18px;
  bottom: -18px;
  width: 2px;
  background: var(--border);
}
.timeline-dot {
  width: 14px;
  height: 14px;
  border-radius: 50%;
  background: var(--primary);
  margin-top: 3px;
  z-index: 1;
}
.timeline-head {
  display: flex;
  justify-content: space-between;
  gap: 10px;
  flex-wrap: wrap;
}
.timeline-comment {
  margin-top: 4px;
  background: var(--surface-2);
  padding: 8px 10px;
  border-radius: 8px;
}

/* ---- Toasts ---- */
.toast-stack {
  position: fixed;
  top: 20px;
  right: 20px;
  display: flex;
  flex-direction: column;
  gap: 10px;
  z-index: 1000;
}
.toast {
  padding: 12px 16px;
  border-radius: 8px;
  color: #fff;
  font-weight: 500;
  box-shadow: var(--shadow);
  min-width: 240px;
  animation: slide-in 0.2s ease;
}
.toast-success {
  background: #16a34a;
}
.toast-error {
  background: #dc2626;
}
.toast-info {
  background: #2563eb;
}
@keyframes slide-in {
  from {
    transform: translateX(20px);
    opacity: 0;
  }
  to {
    transform: translateX(0);
    opacity: 1;
  }
}

/* ---- Responsive ---- */
@media (max-width: 900px) {
  .app-shell {
    grid-template-columns: 1fr;
  }
  .sidebar {
    position: static;
    height: auto;
    flex-direction: row;
    flex-wrap: wrap;
    align-items: center;
  }
}

```



```

}
.nav {
  flex-direction: row;
  flex-wrap: wrap;
  flex: 1 1 100%;
}
.nav-heading {
  display: none;
}
.sidebar-footer {
  border: none;
  flex-direction: row;
  align-items: center;
}
}
.detail-grid {
  grid-template-columns: 1fr;
}
}
}
@media (max-width: 640px) {
  .grid-2 {
    grid-template-columns: 1fr;
  }
  .detail-meta {
    grid-template-columns: 1fr;
  }
  .content {
    padding: 20px 16px;
  }
}
}

```

UniversityMaintenance.Web/src/main.tsx

TSX

```

import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import { BrowserRouter } from 'react-router-dom';
import App from './App';
import { AuthProvider } from './auth/AuthContext';
import { ToastProvider } from './components/Toast';
import './index.css';

createRoot(document.getElementById('root')!).render(
  <StrictMode>
    <BrowserRouter>
      <ToastProvider>
        <AuthProvider>
          <App />
        </AuthProvider>
      </ToastProvider>
    </BrowserRouter>
  </StrictMode>,
);

```

UniversityMaintenance.Web/src/pages/DashboardPage.tsx

TSX

```

import { useEffect, useState } from 'react';
import { Link } from 'react-router-dom';
import { errorMessage } from '../api/client';
import { reportsApi, requestsApi } from '../api/endpoints';
import { useAuth } from '../auth/AuthContext';
import { useToast } from '../components/Toast';
import { StatusBadge } from '../components/StatusBadge';
import { Roles, STATUS_LABELS, type ReportSummary, type ServiceRequestDto } from '../types';

export function DashboardPage() {
  const { user, hasRole } = useAuth();
  const toast = useToast();
  const [summary, setSummary] = useState<ReportSummary | null>(null);
  const [recent, setRecent] = useState<ServiceRequestDto[]>([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    (async () => {
      try {
        const recentPage = await requestsApi.getPaged({ page: 1, pageSize: 5 });
        setRecent(recentPage.items);
        if (hasRole(Roles.Admin)) setSummary(await reportsApi.summary());
      } catch (err) {
        toast.error(errorMessage(err));
      } finally {
        setLoading(false);
      }
    })();
    // eslint-disable-next-line react-hooks/exhaustive-deps
  }, []);

  return (
    <div className="page">
      <header className="page-head">
        <div>
          <h1>Welcome, {user?.firstName} 🍌 </h1>
          <p className="muted">

```

```

    {hasRole(Roles.Student) && 'Submit new maintenance requests and track their progress.'}
    {hasRole(Roles.Officer) && 'Here are the jobs assigned to you.'}
    {hasRole(Roles.Admin) && 'Overview of maintenance activity across the university.'}
  </p>
</div>
{hasRole(Roles.Student) && (
  <Link to="/requests/new" className="btn btn-primary">
    + New Request
  </Link>
)}
</header>

{hasRole(Roles.Admin) && summary && (
  <section className="stat-grid">
    <StatCard label="Total Requests" value={summary.totalRequests} accent="blue" />
    <StatCard label="Total Users" value={summary.totalUsers} accent="violet" />
    <StatCard
      label="Completed"
      value={summary.requestsByStatus['Completed'] ?? 0}
      accent="green"
    />
    <StatCard
      label="In Progress"
      value={summary.requestsByStatus['InProgress'] ?? 0}
      accent="amber"
    />
  </section>
)}

{hasRole(Roles.Admin) && summary && (
  <section className="card">
    <h2 className="card-title">Requests by status</h2>
    <div className="bar-list">
      {Object.entries(STATUS_LABELS).map(([key, label]) => {
        const count = summary.requestsByStatus[label.replace(' ', '')] ?? 0;
        const pct = summary.totalRequests
          ? Math.round((count / summary.totalRequests) * 100)
          : 0;
        return (
          <div key={key} className="bar-row">
            <span className="bar-label">{label}</span>
            <div className="bar-track">
              <div className="bar-fill" style={{ width: `${pct}%` }} />
            </div>
            <span className="bar-value">{count}</span>
          </div>
        );
      })}
    </div>
  </section>
)}

<section className="card">
  <div className="card-head">
    <h2 className="card-title">Recent requests</h2>
    <Link to="/requests" className="link">
      View all →
    </Link>
  </div>
  {loading ? (
    <p className="muted">Loading...</p>
  ) : recent.length === 0 ? (
    <p className="muted">No requests yet.</p>
  ) : (
    <ul className="recent-list">
      {recent.map((r) => (
        <li key={r.id}>
          <Link to={`/requests/${r.id}`} className="recent-item">
            <div>
              <div className="recent-title">{r.title}</div>
              <div className="muted small">
                {r.categoryName} · {r.location}
              </div>
            </div>
            <StatusBadge status={r.status} />
          </Link>
        </li>
      ))}
    </ul>
  )}
</section>
</div>
);
}

function StatCard({ label, value, accent }: { label: string; value: number; accent: string }) {
  return (
    <div className={`stat-card accent-${accent}`}>
      <div className="stat-value">{value}</div>
      <div className="stat-label">{label}</div>
    </div>
  );
}

```

```

import { render, screen, waitFor } from '@testing-library/react';
import userEvent from '@testing-library/user-event';
import { BrowserRouter } from 'react-router-dom';
import { beforeEach, describe, expect, it, vi } from 'vitest';
import { LoginPage } from '../LoginPage';
import { AuthProvider } from '../auth/AuthContext';
import { ToastProvider } from '../components/Toast';
import { authApi } from '../api/endpoints';

vi.mock('../api/endpoints', () => ({
  authApi: { login: vi.fn(), register: vi.fn() },
}));

function renderLogin() {
  return render(
    <BrowserRouter>
      <ToastProvider>
        <AuthProvider>
          <LoginPage />
        </AuthProvider>
      </ToastProvider>
    </BrowserRouter>,
  );
}

describe('LoginPage', () => {
  beforeEach(() => vi.clearAllMocks());

  it('shows a validation error and does not call the API when the email is invalid', async () => {
    renderLogin();
    await userEvent.type(screen.getByLabelText(/email/i), 'not-an-email');
    await userEvent.type(screen.getByLabelText(/password/i), 'secret');
    await userEvent.click(screen.getByRole('button', { name: /sign in/i }));

    expect(await screen.findByText(/valid email/i)).toBeInTheDocument();
    expect(authApi.login).not.toHaveBeenCalled();
  });

  it('calls the login API with valid credentials', async () => {
    (authApi.login as ReturnType<typeof vi.fn>).mockResolvedValue({
      token: 't',
      expiresAt: new Date().toISOString(),
      user: { id: '1', firstName: 'A', lastName: 'B', fullName: 'A B', email: 'a@x.edu', role: 'Student', createdAt: '' },
    });

    renderLogin();
    await userEvent.type(screen.getByLabelText(/email/i), 'a@x.edu');
    await userEvent.type(screen.getByLabelText(/password/i), 'secret1');
    await userEvent.click(screen.getByRole('button', { name: /sign in/i }));

    await waitFor(() => expect(authApi.login).toHaveBeenCalledWith('a@x.edu', 'secret1'));
  });
});

```

```

import { zodResolver } from '@hookform/resolvers/zod';
import { useForm } from 'react-hook-form';
import { Link, useNavigate } from 'react-router-dom';
import { z } from 'zod';
import { errorMessage } from '../api/client';
import { useAuth } from '../auth/AuthContext';
import { useToast } from '../components/Toast';

const schema = z.object({
  email: z.string().email('Enter a valid email'),
  password: z.string().min(1, 'Password is required'),
});
type FormValues = z.infer<typeof schema>;

export function LoginPage() {
  const { login } = useAuth();
  const toast = useToast();
  const navigate = useNavigate();
  const {
    register,
    handleSubmit,
    formState: { errors, isSubmitting },
  } = useForm<FormValues>({ resolver: zodResolver(schema) });

  const onSubmit = async (values: FormValues) => {
    try {
      await login(values.email, values.password);
      toast.success('Welcome back!');
      navigate('/dashboard');
    } catch (err) {
      toast.error(errorMessage(err, 'Login failed'));
    }
  };
}

```

```

return (
  <div className="auth-page">
    <div className="auth-card">
      <div className="auth-brand">
        <span className="brand-mark">UM</span>
        <h1>University Maintenance</h1>
        <p className="muted">Sign in to submit and track service requests.</p>
      </div>

      <form onSubmit={handleSubmit(onSubmit)} className="form" noValidate>
        <label className="field">
          <span>Email</span>
          <input type="email" autoComplete="email" {...register('email')} />
          {errors.email && <small className="err">{errors.email.message}</small>}
        </label>

        <label className="field">
          <span>Password</span>
          <input type="password" autoComplete="current-password" {...register('password')} />
          {errors.password && <small className="err">{errors.password.message}</small>}
        </label>

        <button className="btn btn-primary btn-block" type="submit" disabled={isSubmitting}>
          {isSubmitting ? 'Signing in...' : 'Sign in'}
        </button>
      </form>

      <p className="auth-alt">
        No account? <Link to="/register">Create one</Link>
      </p>
    </div>
  </div>
);
}

```

UniversityMaintenance.Web/src/pages/RegisterPage.tsx

TSX

```

import { zodResolver } from '@hookform/resolvers/zod';
import { useForm } from 'react-hook-form';
import { Link, useNavigate } from 'react-router-dom';
import { z } from 'zod';
import { errorMessage } from '../api/client';
import { useAuth } from '../auth/AuthContext';
import { useToast } from '../components/Toast';

const schema = z.object({
  firstName: z.string().min(1, 'First name is required'),
  lastName: z.string().min(1, 'Last name is required'),
  email: z.string().email('Enter a valid email'),
  password: z.string().min(6, 'Password must be at least 6 characters'),
});
type FormValues = z.infer<typeof schema>;

export function RegisterPage() {
  const { register: registerUser } = useAuth();
  const toast = useToast();
  const navigate = useNavigate();
  const {
    register,
    handleSubmit,
    formState: { errors, isSubmitting },
  } = useForm<FormValues>({ resolver: zodResolver(schema) });

  const onSubmit = async (values: FormValues) => {
    try {
      await registerUser(values.firstName, values.lastName, values.email, values.password);
      toast.success('Account created!');
      navigate('/dashboard');
    } catch (err) {
      toast.error(errorMessage(err, 'Registration failed'));
    }
  };

  return (
    <div className="auth-page">
      <div className="auth-card">
        <div className="auth-brand">
          <span className="brand-mark">UM</span>
          <h1>Create your account</h1>
          <p className="muted">Register as a student or staff member.</p>
        </div>

        <form onSubmit={handleSubmit(onSubmit)} className="form" noValidate>
          <div className="grid-2">
            <label className="field">
              <span>First name</span>
              <input {...register('firstName')} />
              {errors.firstName && <small className="err">{errors.firstName.message}</small>}
            </label>
            <label className="field">
              <span>Last name</span>
              <input {...register('lastName')} />
              {errors.lastName && <small className="err">{errors.lastName.message}</small>}
            </label>

```

```

        </label>
      </div>

      <label className="field">
        <span>Email</span>
        <input type="email" autoComplete="email" {...register('email')} />
        {errors.email && <small className="err">{errors.email.message}</small>}
      </label>

      <label className="field">
        <span>Password</span>
        <input type="password" autoComplete="new-password" {...register('password')} />
        {errors.password && <small className="err">{errors.password.message}</small>}
      </label>

      <button className="btn btn-primary btn-block" type="submit" disabled={isSubmitting}>
        {isSubmitting ? 'Creating...' : 'Create account'}
      </button>
    </form>

    <p className="auth-alt">
      Already registered? <Link to="/login">Sign in</Link>
    </p>
  </div>
</div>
);
}

```

UniversityMaintenance.Web/src/pages/RequestDetailPage.tsx

TSX

```

import { useCallback, useEffect, useState } from 'react';
import { useNavigate, useParams } from 'react-router-dom';
import { errorMessage, imageUrl } from '../api/client';
import { assignmentsApi, requestsApi, usersApi } from '../api/endpoints';
import { useAuth } from '../auth/AuthContext';
import { PriorityBadge, StatusBadge } from '../components/StatusBadge';
import { useToast } from '../components/Toast';
import {
  Roles,
  STATUS_LABELS,
  RequestStatus,
  type ServiceRequestDto,
  type StatusUpdateDto,
  type UserDto,
} from '../types';

export function RequestDetailPage() {
  const { id = '' } = useParams();
  const { hasRole } = useAuth();
  const toast = useToast();
  const navigate = useNavigate();

  const [request, setRequest] = useState<ServiceRequestDto | null>(null);
  const [history, setHistory] = useState<StatusUpdateDto[]>([]);
  const [officers, setOfficers] = useState<UserDto[]>([]);
  const [loading, setLoading] = useState(true);

  const [newStatus, setNewStatus] = useState<string>('');
  const [comment, setComment] = useState('');
  const [officerId, setOfficerId] = useState('');
  const [notes, setNotes] = useState('');

  const load = useCallback(async () => {
    try {
      const [req, hist] = await Promise.all([
        requestsApi.getById(id),
        requestsApi.getHistory(id),
      ]);
      setRequest(req);
      setHistory(hist);
      setNewStatus(String(req.status));
    } catch (err) {
      toast.error(errorMessage(err));
    } finally {
      setLoading(false);
    }
    // eslint-disable-next-line react-hooks/exhaustive-deps
  }, [id]);

  useEffect(() => {
    load();
    if (hasRole(Roles.Admin)) usersApi.getOfficers().then(setOfficers).catch(() => undefined);
    // eslint-disable-next-line react-hooks/exhaustive-deps
  }, [id]);

  const canManage = hasRole(Roles.Officer, Roles.Admin);

  const submitStatus = async () => {
    try {
      await requestsApi.updateStatus(id, Number(newStatus), comment || undefined);
      toast.success('Status updated');
      setComment('');
      load();
    }
  }

```

```

    } catch (err) {
      toast.error(errorMessage(err));
    }
  };

const submitAssign = async () => {
  if (!officerId) return toast.error('Select an officer');
  try {
    await assignmentsApi.assign(id, officerId, notes || undefined);
    toast.success('Request assigned');
    setNotes('');
    setOfficerId('');
    load();
  } catch (err) {
    toast.error(errorMessage(err));
  }
};

if (loading) return <div className="page">Loading...</div>;
if (!request) return <div className="page">Request not found.</div>;

const img = imageUrl(request.evidenceImagePath);

return (
  <div className="page">
    <button className="link back" onClick={() => navigate(-1)}>
      ← Back
    </button>

    <div className="detail-grid">
      <div className="detail-main">
        <div className="card">
          <div className="detail-head">
            <div>
              <h1>{request.title}</h1>
              <p className="muted">
                {request.categoryName} · {request.location}
              </p>
            </div>
            <div className="badge-col">
              <StatusBadge status={request.status} />
              <PriorityBadge priority={request.priority} />
            </div>
          </div>

          <p className="detail-desc">{request.description}</p>

          <dl className="detail-meta">
            <div>
              <dt>Requester</dt>
              <dd>{request.requesterName}</dd>
            </div>
            <div>
              <dt>Assigned officer</dt>
              <dd>{request.assignedOfficerName ?? '-'}</dd>
            </div>
            <div>
              <dt>Created</dt>
              <dd>{new Date(request.createdAt).toLocaleString()}</dd>
            </div>
          </dl>

          {img && (
            <div className="evidence">
              <span className="muted small">Evidence</span>
              <a href={img} target="_blank" rel="noreferrer">
                <img src={img} alt="Fault evidence" />
              </a>
            </div>
          )}
        </div>
      </div>

      <div className="card">
        <h2 className="card-title">Activity timeline</h2>
        <ol className="timeline">
          {history.map((h) => (
            <li key={h.id}>
              <span className="timeline-dot" />
              <div>
                <div className="timeline-head">
                  {h.oldStatus === h.newStatus ? (
                    <strong>{STATUS_LABELS[h.newStatus]}</strong>
                  ) : (
                    <strong>
                      {STATUS_LABELS[h.oldStatus]} → {STATUS_LABELS[h.newStatus]}
                    </strong>
                  )}
                <span className="muted small">
                  {new Date(h.createdAt).toLocaleString()}
                </span>
              </div>
              <div className="muted small">by {h.changedByName}</div>
              {h.comment && <div className="timeline-comment">{h.comment}</div>}
            </li>
          )}
        </ol>
      </div>
    </div>
  );

```

```

    ))}
  </ol>
</div>
</div>

{canManage && (
  <aside className="detail-side">
    <div className="card">
      <h2 className="card-title">Update status</h2>
      <label className="field">
        <span>New status</span>
        <select value={newStatus} onChange={(e) => setNewStatus(e.target.value)}>
          {Object.entries(STATUS_LABELS).map(([value, label]) => (
            <option key={value} value={value}>
              {label}
            </option>
          ))}
        </select>
      </label>
      <label className="field">
        <span>Comment</span>
        <textarea
          rows={3}
          value={comment}
          onChange={(e) => setComment(e.target.value)}
          placeholder="Optional note..."
        />
      </label>
      <button className="btn btn-primary btn-block" onClick={submitStatus}>
        Save status
      </button>
    </div>

    {hasRole(Roles.Admin) && (
      <div className="card">
        <h2 className="card-title">Assign officer</h2>
        <label className="field">
          <span>Officer</span>
          <select value={officerId} onChange={(e) => setOfficerId(e.target.value)}>
            <option value="">Select...</option>
            {officers.map((o) => (
              <option key={o.id} value={o.id}>
                {o.fullName}
              </option>
            ))}
          </select>
        </label>
        <label className="field">
          <span>Notes</span>
          <textarea
            rows={2}
            value={notes}
            onChange={(e) => setNotes(e.target.value)}
            placeholder="Optional..."
          />
        </label>
        <button
          className="btn btn-secondary btn-block"
          onClick={submitAssign}
          disabled={request.status === RequestStatus.Completed}
        >
          Assign request
        </button>
      </div>
    )}
  </aside>
)}
</div>
</div>
);
}

```

UniversityMaintenance.Web/src/pages/RequestsListPage.tsx

TSX

```

import { useEffect, useState } from 'react';
import { Link } from 'react-router-dom';
import { errorMessage } from '../api/client';
import { categoriesApi, requestsApi } from '../api/endpoints';
import { useAuth } from '../auth/AuthContext';
import { Pagination } from '../components/Pagination';
import { PriorityBadge, StatusBadge } from '../components/StatusBadge';
import { useToast } from '../components/Toast';
import {
  Roles,
  STATUS_LABELS,
  type CategoryDto,
  type PagedResult,
  type ServiceRequestDto,
} from '../types';

export function RequestsListPage() {
  const { hasRole } = useAuth();
  const toast = useToast();

```

```

const [data, setData] = useState<PagedResult<ServiceRequestDto> | null>(null);
const [categories, setCategories] = useState<CategoryDto[]>([]);
const [loading, setLoading] = useState(true);

const [search, setSearch] = useState('');
const [status, setStatus] = useState<string>('');
const [categoryId, setCategoryId] = useState<string>('');
const [page, setPage] = useState(1);

const title = hasRole(Roles.Student)
  ? 'My Requests'
  : hasRole(Roles.Officer)
    ? 'Assigned Jobs'
    : 'All Requests';

useEffect(() => {
  categoriesApi.getAll().then(setCategories).catch(() => undefined);
}, []);

useEffect(() => {
  setLoading(true);
  requestsApi
    .getPaged({
      search: search || undefined,
      status: status === '' ? undefined : Number(status),
      categoryId: categoryId || undefined,
      page,
      pageSize: 8,
    })
    .then(setData)
    .catch((err) => toast.error(errorMessage(err)))
    .finally(() => setLoading(false));
  // eslint-disable-next-line react-hooks/exhaustive-deps
}, [search, status, categoryId, page]);

return (
  <div className="page">
    <header className="page-head">
      <h1>{title}</h1>
      {hasRole(Roles.Student) && (
        <Link to="/requests/new" className="btn btn-primary">
          + New Request
        </Link>
      )}
    </header>

    <div className="filters card">
      <input
        className="filter-search"
        placeholder="Search title, description, location..."
        value={search}
        onChange={(e) => {
          setPage(1);
          setSearch(e.target.value);
        }}
      />
      <select
        value={status}
        onChange={(e) => {
          setPage(1);
          setStatus(e.target.value);
        }}
      >
        <option value="">All statuses</option>
        {Object.entries(STATUS_LABELS).map(([value, label]) => (
          <option key={value} value={value}>
            {label}
          </option>
        ))}
      </select>
      <select
        value={categoryId}
        onChange={(e) => {
          setPage(1);
          setCategoryId(e.target.value);
        }}
      >
        <option value="">All categories</option>
        {categories.map((c) => (
          <option key={c.id} value={c.id}>
            {c.name}
          </option>
        ))}
      </select>
    </div>

    <div className="card">
      {loading ? (
        <p className="muted">Loading...</p>
      ) : !data || data.items.length === 0 ? (
        <p className="muted empty">No requests match your filters.</p>
      ) : (
        <div className="table-wrap">
          <table className="table">
            <thead>

```



```

        <tr>
          <th>Title</th>
          <th>Category</th>
          <th>Location</th>
          <th>Priority</th>
          <th>Status</th>
          {!hasRole(Roles.Student) && <th>Requester</th>}
          <th></th>
        </tr>
      </thead>
      <tbody>
        {data.items.map((r) => (
          <tr key={r.id}>
            <td className="strong">{r.title}</td>
            <td>{r.categoryName}</td>
            <td>{r.location}</td>
            <td>
              <PriorityBadge priority={r.priority} />
            </td>
            <td>
              <StatusBadge status={r.status} />
            </td>
            {!hasRole(Roles.Student) && <td>{r.requesterName}</td>}
            <td>
              <Link to={` /requests/${r.id}`} className="link">
                View
              </Link>
            </td>
          </tr>
        ))}
      </tbody>
    </table>
  </div>
)}

{data && (
  <Pagination
    page={data.page}
    totalPages={data.totalPages}
    totalCount={data.totalCount}
    onPage={setPage}
  />
)}
</div>
</div>
);
}

```

UniversityMaintenance.Web/src/pages/SubmitRequestPage.tsx

TSX

```

import { zodResolver } from '@hookform/resolvers/zod';
import { useEffect, useState } from 'react';
import { useForm } from 'react-hook-form';
import { useNavigate } from 'react-router-dom';
import { z } from 'zod';
import { errorMessage } from '../api/client';
import { categoriesApi, requestsApi } from '../api/endpoints';
import { useToast } from '../components/Toast';
import { Priority, type CategoryDto } from '../types';

const schema = z.object({
  title: z.string().min(1, 'Title is required').max(200),
  categoryId: z.string().min(1, 'Choose a category'),
  location: z.string().min(1, 'Location is required'),
  priority: z.string(),
  description: z.string().min(1, 'Description is required').max(2000),
});
type FormValues = z.infer<typeof schema>;

export function SubmitRequestPage() {
  const toast = useToast();
  const navigate = useNavigate();
  const [categories, setCategories] = useState<CategoryDto[]>([]);
  const [file, setFile] = useState<File | null>(null);

  const {
    register,
    handleSubmit,
    formState: { errors, isSubmitting },
  } = useForm<FormValues>({
    resolver: zodResolver(schema),
    defaultValues: { priority: String(Priority.Medium) },
  });

  useEffect(() => {
    categoriesApi.getAll().then(setCategories).catch((err) => toast.error(errorMessage(err)));
    // eslint-disable-next-line react-hooks/exhaustive-deps
  }, []);

  const onSubmit = async (values: FormValues) => {
    try {
      const form = new FormData();
      form.append('Title', values.title);
    }
  }
}

```

```

form.append('Description', values.description);
form.append('Location', values.location);
form.append('Priority', values.priority);
form.append('CategoryId', values.categoryId);
if (file) form.append('evidence', file);

const created = await requestsApi.create(form);
toast.success('Request submitted!');
navigate(`/requests/${created.id}`);
} catch (err) {
  toast.error(errorMessage(err, 'Could not submit request'));
}
};

return (
  <div className="page narrow">
    <header className="page-head">
      <h1>Submit a Service Request</h1>
    </header>

    <form onSubmit={handleSubmit(onSubmit)} className="card form" noValidate>
      <label className="field">
        <span>Title</span>
        <input placeholder="e.g. Broken socket in Room 12" {...register('title')} />
        {errors.title && <small className="err">{errors.title.message}</small>}
      </label>

      <div className="grid-2">
        <label className="field">
          <span>Category</span>
          <select {...register('categoryId')}>
            <option value="">Select...</option>
            {categories.map((c) => (
              <option key={c.id} value={c.id}>
                {c.name}
              </option>
            ))}
          </select>
          {errors.categoryId && <small className="err">{errors.categoryId.message}</small>}
        </label>

        <label className="field">
          <span>Priority</span>
          <select {...register('priority')}>
            <option value={Priority.Low}>Low</option>
            <option value={Priority.Medium}>Medium</option>
            <option value={Priority.High}>High</option>
          </select>
        </label>
      </div>

      <label className="field">
        <span>Location</span>
        <input placeholder="e.g. Block A, Room 12" {...register('location')} />
        {errors.location && <small className="err">{errors.location.message}</small>}
      </label>

      <label className="field">
        <span>Description</span>
        <textarea rows={5} placeholder="Describe the fault..." {...register('description')} />
        {errors.description && <small className="err">{errors.description.message}</small>}
      </label>

      <label className="field">
        <span>Evidence photo (optional)</span>
        <input
          type="file"
          accept="image/*"
          onChange={(e) => setFile(e.target.files?.[0] ?? null)}
        />
        {file && <small className="muted">{file.name}</small>}
      </label>

      <div className="form-actions">
        <button type="button" className="btn btn-ghost" onClick={() => navigate(-1)}>
          Cancel
        </button>
        <button type="submit" className="btn btn-primary" disabled={isSubmitting}>
          {isSubmitting ? 'Submitting...' : 'Submit Request'}
        </button>
      </div>
    </form>
  </div>
);
}

```

```

export function CategoriesPage() {
  const toast = useToast();
  const [categories, setCategories] = useState<CategoryDto[]>([]);
  const [name, setName] = useState('');
  const [description, setDescription] = useState('');

  const load = () => {
    categoriesApi.getAll().then(setCategories).catch((err) => toast.error(errorMessage(err)));
  };

  useEffect(() => {
    load();
    // eslint-disable-next-line react-hooks/exhaustive-deps
  }, []);

  const add = async (e: React.FormEvent) => {
    e.preventDefault();
    try {
      await categoriesApi.create(name, description || undefined);
      toast.success('Category added');
      setName('');
      setDescription('');
      load();
    } catch (err) {
      toast.error(errorMessage(err));
    }
  };

  const remove = async (id: string) => {
    if (!confirm('Delete this category?')) return;
    try {
      await categoriesApi.remove(id);
      toast.success('Category deleted');
      load();
    } catch (err) {
      toast.error(errorMessage(err));
    }
  };

  return (
    <div className="page">
      <header className="page-head">
        <h1>Categories</h1>
      </header>

      <form onSubmit={add} className="card form">
        <div className="grid-2">
          <label className="field">
            <span>Name</span>
            <input required value={name} onChange={(e) => setName(e.target.value)} />
          </label>
          <label className="field">
            <span>Description</span>
            <input value={description} onChange={(e) => setDescription(e.target.value)} />
          </label>
        </div>
        <div className="form-actions">
          <button className="btn btn-primary" type="submit">
            Add category
          </button>
        </div>
      </form>

      <div className="card">
        <div className="table-wrap">
          <table className="table">
            <thead>
              <tr>
                <th>Name</th>
                <th>Description</th>
                <th></th>
              </tr>
            </thead>
            <tbody>
              {categories.map((c) => (
                <tr key={c.id}>
                  <td className="strong">{c.name}</td>
                  <td>{c.description ?? '-'}</td>
                  <td>
                    <button className="link danger" onClick={() => remove(c.id)}>
                      Delete
                    </button>
                  </td>
                </tr>
              ))}
            </tbody>
          </table>
        </div>
      </div>
    </div>
  );
}

```

```

import { useEffect, useState } from 'react';
import { errorMessage } from '../../../api/client';
import { reportsApi } from '../../../api/endpoints';
import { useToast } from '../../../components/Toast';
import type { ReportSummary } from '../../../types';

export function ReportsPage() {
  const toast = useToast();
  const [summary, setSummary] = useState<ReportSummary | null>(null);

  useEffect(() => {
    reportsApi.summary().then(setSummary).catch((err) => toast.error(errorMessage(err)));
    // eslint-disable-next-line react-hooks/exhaustive-deps
  }, []);

  const exportCsv = () => {
    if (!summary) return;
    const rows: string[] = ['Metric,Value', `Total Requests,${summary.totalRequests}`, `Total Users,${summary.totalUsers}`];
    const section = (title: string, obj: Record<string, number>) => {
      rows.push('', title);
      Object.entries(obj).forEach(([k, v]) => rows.push(`${k},${v}`));
    };
    section('By Status', summary.requestsByStatus);
    section('By Category', summary.requestsByCategory);
    section('By Priority', summary.requestsByPriority);

    const blob = new Blob([rows.join('\n')], { type: 'text/csv' });
    const url = URL.createObjectURL(blob);
    const a = document.createElement('a');
    a.href = url;
    a.download = 'maintenance-report.csv';
    a.click();
    URL.revokeObjectURL(url);
  };

  if (!summary) return <div className="page">Loading...</div>;

  return (
    <div className="page">
      <header className="page-head">
        <h1>Reports</h1>
        <button className="btn btn-secondary" onClick={exportCsv}>
          Export CSV
        </button>
      </header>

      <section className="stat-grid">
        <div className="stat-card accent-blue">
          <div className="stat-value">{summary.totalRequests}</div>
          <div className="stat-label">Total Requests</div>
        </div>
        <div className="stat-card accent-violet">
          <div className="stat-value">{summary.totalUsers}</div>
          <div className="stat-label">Total Users</div>
        </div>
      </section>

      <div className="report-cols">
        <BreakdownCard title="By Status" data={summary.requestsByStatus} />
        <BreakdownCard title="By Category" data={summary.requestsByCategory} />
        <BreakdownCard title="By Priority" data={summary.requestsByPriority} />
      </div>
    </div>
  );
}

function BreakdownCard({ title, data }: { title: string; data: Record<string, number> }) {
  const total = Object.values(data).reduce((a, b) => a + b, 0);
  return (
    <div className="card">
      <h2 className="card-title">{title}</h2>
      {Object.keys(data).length === 0 ? (
        <p className="muted">No data.</p>
      ) : (
        <div className="bar-list">
          {Object.entries(data).map(([label, count]) => (
            <div key={label} className="bar-row">
              <span className="bar-label">{label}</span>
              <div className="bar-track">
                <div
                  className="bar-fill"
                  style={{ width: `${total ? Math.round((count / total) * 100) : 0}%` }}
                </div>
              </div>
              <span className="bar-value">{count}</span>
            </div>
          ))}
        </div>
      )}
    </div>
  );
}

```

```

import { useEffect, useState } from 'react';
import { errorMessage } from '../../../api/client';
import { usersApi } from '../../../api/endpoints';
import { Pagination } from '../../../components/Pagination';
import { useToast } from '../../../components/Toast';
import { useAuth } from '../../../auth/AuthContext';
import { Roles, type PagedResult, type UserDto } from '../../../types';

export function UsersPage() {
  const toast = useToast();
  const { user: me } = useAuth();
  const [data, setData] = useState<PagedResult<UserDto> | null>(null);
  const [search, setSearch] = useState('');
  const [page, setPage] = useState(1);
  const [showForm, setShowForm] = useState(false);

  const [form, setForm] = useState({
    firstName: '',
    lastName: '',
    email: '',
    password: '',
    role: Roles.Officer as string,
  });

  const load = () => {
    usersApi
      .getPaged(search, page, 8)
      .then(setData)
      .catch((err) => toast.error(errorMessage(err)));
  };

  useEffect(load, [search, page]); // eslint-disable-line react-hooks/exhaustive-deps

  const createUser = async (e: React.FormEvent) => {
    e.preventDefault();
    try {
      await usersApi.create(form);
      toast.success('User created');
      setForm({ firstName: '', lastName: '', email: '', password: '', role: Roles.Officer });
      setShowForm(false);
      load();
    } catch (err) {
      toast.error(errorMessage(err));
    }
  };

  const removeUser = async (id: string) => {
    if (!confirm('Delete this user?')) return;
    try {
      await usersApi.remove(id);
      toast.success('User deleted');
      load();
    } catch (err) {
      toast.error(errorMessage(err));
    }
  };

  return (
    <div className="page">
      <header className="page-head">
        <h1>Users</h1>
        <button className="btn btn-primary" onClick={() => setShowForm((s) => !s)}>
          {showForm ? 'Close' : 'New User'}
        </button>
      </header>

      {showForm && (
        <form onSubmit={createUser} className="card form">
          <div className="grid-2">
            <label className="field">
              <span>First name</span>
              <input
                required
                value={form.firstName}
                onChange={(e) => setForm({ ...form, firstName: e.target.value })}
              />
            </label>
            <label className="field">
              <span>Last name</span>
              <input
                required
                value={form.lastName}
                onChange={(e) => setForm({ ...form, lastName: e.target.value })}
              />
            </label>
          </div>
          <div className="grid-2">
            <label className="field">
              <span>Email</span>
              <input
                type="email"
                required

```

```

        value={form.email}
        onChange={(e) => setForm({ ...form, email: e.target.value })}
      />
    </label>
    <label className="field">
      <span>Password</span>
      <input
        type="password"
        required
        minLength={6}
        value={form.password}
        onChange={(e) => setForm({ ...form, password: e.target.value })}
      />
    </label>
  </div>
  <label className="field">
    <span>Role</span>
    <select value={form.role} onChange={(e) => setForm({ ...form, role: e.target.value })}>
      <option value={Roles.Officer}>Officer</option>
      <option value={Roles.Admin}>Admin</option>
      <option value={Roles.Student}>Student</option>
    </select>
  </label>
  <div className="form-actions">
    <button className="btn btn-primary" type="submit">
      Create user
    </button>
  </div>
</form>
)}

<div className="filters card">
  <input
    className="filter-search"
    placeholder="Search name or email..."
    value={search}
    onChange={(e) => {
      setPage(1);
      setSearch(e.target.value);
    }}
  />
</div>

<div className="card">
  <div className="table-wrap">
    <table className="table">
      <thead>
        <tr>
          <th>Name</th>
          <th>Email</th>
          <th>Role</th>
          <th>Joined</th>
          <th></th>
        </tr>
      </thead>
      <tbody>
        {data?.items.map((u) => (
          <tr key={u.id}>
            <td className="strong">{u.fullName}</td>
            <td>{u.email}</td>
            <td>
              <span className="badge badge-role">{u.role}</span>
            </td>
            <td>{new Date(u.createdAt).toLocaleDateString()}</td>
            <td>
              {u.id !== me?.id && (
                <button className="link danger" onClick={() => removeUser(u.id)}>
                  Delete
                </button>
              )}
            </td>
          </tr>
        ))}
      </tbody>
    </table>
  </div>
  {data && (
    <Pagination
      page={data.page}
      totalPages={data.totalPages}
      totalCount={data.totalCount}
      onPage={setPage}
    />
  )}
</div>
</div>
);
}

```

```
// ---- Enums (numeric, matching the API's System.Text.Json serialization) ----
// Modelled as const objects + union types because the project's tsconfig enables
// `erasableSyntaxOnly`, which disallows TS `enum` declarations.
export const Priority = {
  Low: 0,
  Medium: 1,
  High: 2,
} as const;
export type Priority = (typeof Priority)[keyof typeof Priority];

export const RequestStatus = {
  Submitted: 0,
  Assigned: 1,
  InProgress: 2,
  Completed: 3,
  Rejected: 4,
} as const;
export type RequestStatus = (typeof RequestStatus)[keyof typeof RequestStatus];

export const PRIORITY_LABELS: Record<number, string> = {
  [Priority.Low]: 'Low',
  [Priority.Medium]: 'Medium',
  [Priority.High]: 'High',
};

export const STATUS_LABELS: Record<number, string> = {
  [RequestStatus.Submitted]: 'Submitted',
  [RequestStatus.Assigned]: 'Assigned',
  [RequestStatus.InProgress]: 'In Progress',
  [RequestStatus.Completed]: 'Completed',
  [RequestStatus.Rejected]: 'Rejected',
};

export const Roles = {
  Student: 'Student',
  Officer: 'Officer',
  Admin: 'Admin',
} as const;
export type Role = (typeof Roles)[keyof typeof Roles];

// ---- DTOs ----
export interface UserDto {
  id: string;
  firstName: string;
  lastName: string;
  fullName: string;
  email: string;
  role: string;
  createdAt: string;
}

export interface AuthResponse {
  token: string;
  expiresAt: string;
  user: UserDto;
}

export interface CategoryDto {
  id: string;
  name: string;
  description?: string | null;
}

export interface ServiceRequestDto {
  id: string;
  title: string;
  description: string;
  location: string;
  priority: Priority;
  status: RequestStatus;
  evidenceImagePath?: string | null;
  categoryId: string;
  categoryName: string;
  requesterId: string;
  requesterName: string;
  assignedOfficerId?: string | null;
  assignedOfficerName?: string | null;
  createdAt: string;
  updatedAt?: string | null;
}

export interface StatusUpdateDto {
  id: string;
  oldStatus: RequestStatus;
  newStatus: RequestStatus;
  changedById: string;
  changedByName: string;
  comment?: string | null;
  createdAt: string;
}

export interface AssignmentDto {
```

```

id: string;
serviceRequestId: string;
serviceRequestTitle: string;
officerId: string;
officerName: string;
assignedByAdminId: string;
assignedAt: string;
notes?: string | null;
}

export interface PagedResult<T> {
  items: T[];
  page: number;
  pageSize: number;
  totalCount: number;
  totalPages: number;
  hasPrevious: boolean;
  hasNext: boolean;
}

export interface ReportSummary {
  totalRequests: number;
  totalUsers: number;
  requestsByStatus: Record<string, number>;
  requestsByCategory: Record<string, number>;
  requestsByPriority: Record<string, number>;
}

```

Solution & DevOps

UniversityMaintenanceSystem.slnx

XML

```

<Solution>
  <Project Path="UniversityMaintenance.API/UniversityMaintenance.API.csproj" />
  <Project Path="UniversityMaintenance.Application/UniversityMaintenance.Application.csproj" />
  <Project Path="UniversityMaintenance.Domain/UniversityMaintenance.Domain.csproj" />
  <Project Path="UniversityMaintenance.Infrastructure/UniversityMaintenance.Infrastructure.csproj" />
  <Project Path="UniversityMaintenance.Tests/UniversityMaintenance.Tests.csproj" />
</Solution>

```

docker-compose.yml

YAML

```

services:
  db:
    image: postgres:16
    container_name: ums-postgres
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
      POSTGRES_DB: university_maintenance
    ports:
      - "5432:5432"
    volumes:
      - ums_pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 5s
      timeout: 5s
      retries: 5

volumes:
  ums_pgdata:

```

render.yaml

YAML

```

# Render Blueprint – deploys the API as a Docker web service.
# The PostgreSQL database is provisioned separately (Neon free tier or Render Postgres).
services:
  - type: web
    name: university-maintenance-api
    runtime: docker
    dockerfilePath: ./UniversityMaintenance.API/Dockerfile
    dockerContext: .
    plan: free
    healthCheckPath: /swagger/v1/swagger.json
    envVars:
      - key: ASPNETCORE_ENVIRONMENT
        value: Production
      # Set these in the Render dashboard (mark secrets as such):
      - key: ConnectionStrings__DefaultConnection
        sync: false # e.g. Host=...neon.tech;Database=...;Username=...;Password=...;SSL Mode=Require;Trust Server Certificate=true
      - key: Jwt__Key
        sync: false # a long random secret (>= 32 chars)
      - key: Jwt__Issuer
        value: UniversityMaintenance
      - key: Jwt__Audience
        value: UniversityMaintenanceClient

```



```
- key: Cors__AllowedOrigins__0
  sync: false # the deployed frontend origin, e.g. https://your-app.vercel.app
- key: Seed__AdminEmail
  sync: false
- key: Seed__AdminPassword
  sync: false
```

UniversityMaintenance.API/Dockerfile

TXT

```
# ---- Build stage ----
FROM mcr.microsoft.com/dotnet/sdk:10.0 AS build
WORKDIR /src

# Copy csproj files first for restore-layer caching.
COPY UniversityMaintenance.Domain/UniversityMaintenance.Domain.csproj UniversityMaintenance.Domain/
COPY UniversityMaintenance.Application/UniversityMaintenance.Application.csproj UniversityMaintenance.Application/
COPY UniversityMaintenance.Infrastructure/UniversityMaintenance.Infrastructure.csproj UniversityMaintenance.Infrastructure/
COPY UniversityMaintenance.API/UniversityMaintenance.API.csproj UniversityMaintenance.API/
RUN dotnet restore UniversityMaintenance.API/UniversityMaintenance.API.csproj

# Copy the rest and publish.
COPY . .
RUN dotnet publish UniversityMaintenance.API/UniversityMaintenance.API.csproj -c Release -o /app/publish

# ---- Runtime stage ----
FROM mcr.microsoft.com/dotnet/aspnet:10.0 AS runtime
WORKDIR /app
COPY --from=build /app/publish .
# Render/Railway inject PORT at runtime; Program.cs binds to it.
ENV ASPNETCORE_ENVIRONMENT=Production
EXPOSE 8080
ENTRYPOINT ["dotnet", "UniversityMaintenance.API.dll"]
```

UniversityMaintenance.Web/package.json

JSON

```
{
  "name": "universitymaintenance-web",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "tsc -b && vite build",
    "lint": "oxlint",
    "preview": "vite preview",
    "test": "vitest run",
    "test:watch": "vitest"
  },
  "dependencies": {
    "@hookform/resolvers": "^5.4.0",
    "axios": "^1.18.1",
    "react": "^19.2.7",
    "react-dom": "^19.2.7",
    "react-hook-form": "^7.80.0",
    "react-router-dom": "^7.18.1",
    "zod": "^4.4.3"
  },
  "devDependencies": {
    "@testing-library/jest-dom": "^6.9.1",
    "@testing-library/react": "^16.3.2",
    "@testing-library/user-event": "^14.6.1",
    "@types/node": "^24.13.2",
    "@types/react": "^19.2.17",
    "@types/react-dom": "^19.2.3",
    "@vitejs/plugin-react": "^6.0.3",
    "happy-dom": "^20.10.6",
    "oxlint": "^1.71.0",
    "typescript": "~6.0.2",
    "vite": "^8.1.1",
    "vitest": "^3.2.6"
  }
}
```

UniversityMaintenance.Web/vite.config.ts

TS

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'

// https://vite.dev/config/
export default defineConfig({
  plugins: [react()],
})
```

UniversityMaintenance.Web/vitest.config.ts

TS

```
/// <reference types="vitest/config" />
import { defineConfig } from 'vite';
```

```
import react from '@vitejs/plugin-react';

export default defineConfig({
  plugins: [react()],
  test: {
    globals: true,
    environment: 'happy-dom',
    setupFiles: ['./src/test/setup.ts'],
    css: false,
  },
});
```

UniversityMaintenance.Web/vercel.json

JSON

```
{
  "$schema": "https://openapi.vercel.sh/vercel.json",
  "buildCommand": "npm run build",
  "outputDirectory": "dist",
  "rewrites": [{ "source": "/(.*)", "destination": "/index.html" }]
}
```

UniversityMaintenance.Web/index.html

HTML

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="/favicon.svg" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>University Maintenance System</title>
    <meta
      name="description"
      content="Submit, assign and track university maintenance service requests."
    />
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.tsx"></script>
  </body>
</html>
```